- **Efficiency and Accuracy:** Reduces errors and administrative workload while improving turnaround time for submissions and approvals.
- **Enhanced Learning Experience:** Provides students with easier access to learning resources and structured collaboration through automatic grouping.

The proposed LMS transforms the existing portal into a full-featured, user-friendly system that addresses the gaps in the current setup. By automating key academic and administrative processes and providing mobile accessibility, the system will significantly improve efficiency, usability, and the overall learning experience at Bahir Dar University.

2.3 Requirement Analysis

2.3.1 Functional requirement

FREQ-01: User Registration and Login

- **Description:** The system shall allow students, instructors, and administrators to register and log in using unique credentials with secure authentication.
- **Priority:** High

FREQ-02: Profile Management

- **Description:** The system shall allow users to create, view, and update their personal profiles based on their assigned roles.
- **Priority:** High

FREQ-03: Role-Based Access Control

- **Description:** The system shall enforce role-based access, ensuring students, instructors, and administrators can only access authorized features.
- **Priority:** High

FREQ-04: Course Viewing and Enrollment

- **Description:** The system shall allow students to view and enroll in courses associated with their department or major.
- **Priority:** High

FREQ-05: Schedule Management

- **Description:** The system shall display course schedules, lectures, and assessment timelines for students and instructors.
- **Priority:** High

FREQ-06: Access Course Materials

- **Description:** The system shall allow students to access course materials such as documents, videos, and presentations uploaded by instructors.
- **Priority:** High

FREQ-07: Download Learning Resources

- **Description:** The system shall allow students to download course materials for offline use.
- **Priority:** Medium

FREQ-08: Offline Access and Synchronization

- **Description:** The system shall support offline access to downloaded materials and automatically synchronize updates when an internet connection is available.
- **Priority:** Medium

FREQ-09: Assignment Submission

- **Description:** The system shall allow students to submit assignments electronically before specified deadlines.
- **Priority:** High

FREQ-10: Assignment Submission Confirmation

- **Description:** The system shall validate submissions and provide confirmation notifications to students upon successful upload.
- **Priority:** High

FREQ-11: Course Material Management

- **Description:** The system shall allow instructors to upload, update, and delete course materials.
- **Priority:** High

FREQ-12: Course Creation and Management

- **Description:** The system shall allow instructors to create, update, and manage courses, including assessments and learning content.
- **Priority:** High

FREQ-13: Assignment Grading

- **Description:** The system shall allow instructors to grade assignments and provide feedback through the platform.
- **Priority:** High

FREQ-14: Grade Viewing

- **Description:** The system shall allow students to view their grades and feedback for completed assessments.
- **Priority:** Medium

FREQ-15: Student Progress Tracking

- **Description:** The system shall track student progress, including completed assignments, attendance, and assessment performance.
- **Priority:** Medium

FREQ-16: Automatic Group Assignment

- **Description:** The system shall automatically assign students to groups for practical sessions or project-based learning based on defined criteria.
- **Priority:** High

FREQ-17: Group Management

- **Description:** The system shall allow instructors to view, modify, and manage student groups.
- **Priority:** Medium

FREQ-18: Announcement Management

- **Description:** The system shall allow instructors and administrators to create and publish announcements or notifications to students.
- **Priority:** Low

FREQ-19: Announcement Viewing

- **Description:** The system shall allow students to view announcements and notifications relevant to their courses.
- **Priority:** Low

FREQ-20: User Account Administration

- **Description:** The system shall allow administrators to create, update, suspend, or delete user accounts.
- **Priority:** High

FREQ-21: Course and Department Administration

- **Description:** The system shall allow administrators to manage courses, departments, and academic structures.
- **Priority:** High

FREQ-22: Support Request Handling

- **Description:** The system shall allow administrators to receive, track, and resolve user support requests.
- **Priority:** Medium

FREQ-23: System Activity Logging

- **Description:** The system shall log user actions and system events for auditing and monitoring purposes.
- **Priority:** Medium

FREQ-24: System Monitoring

- **Description:** The system shall allow administrators to monitor system performance, usage, and security status.
- **Priority:** Medium

FREQ-25: Report Generation

- **Description:** The system shall allow administrators to generate reports related to user activity, course performance, and system usage.
- **Priority:** Low

FREQ-26: Data Storage and Backup

- **Description:** The system shall securely store, retrieve, and back up user and academic data to prevent data loss.
- **Priority:** High

2.3.2 System Use Case

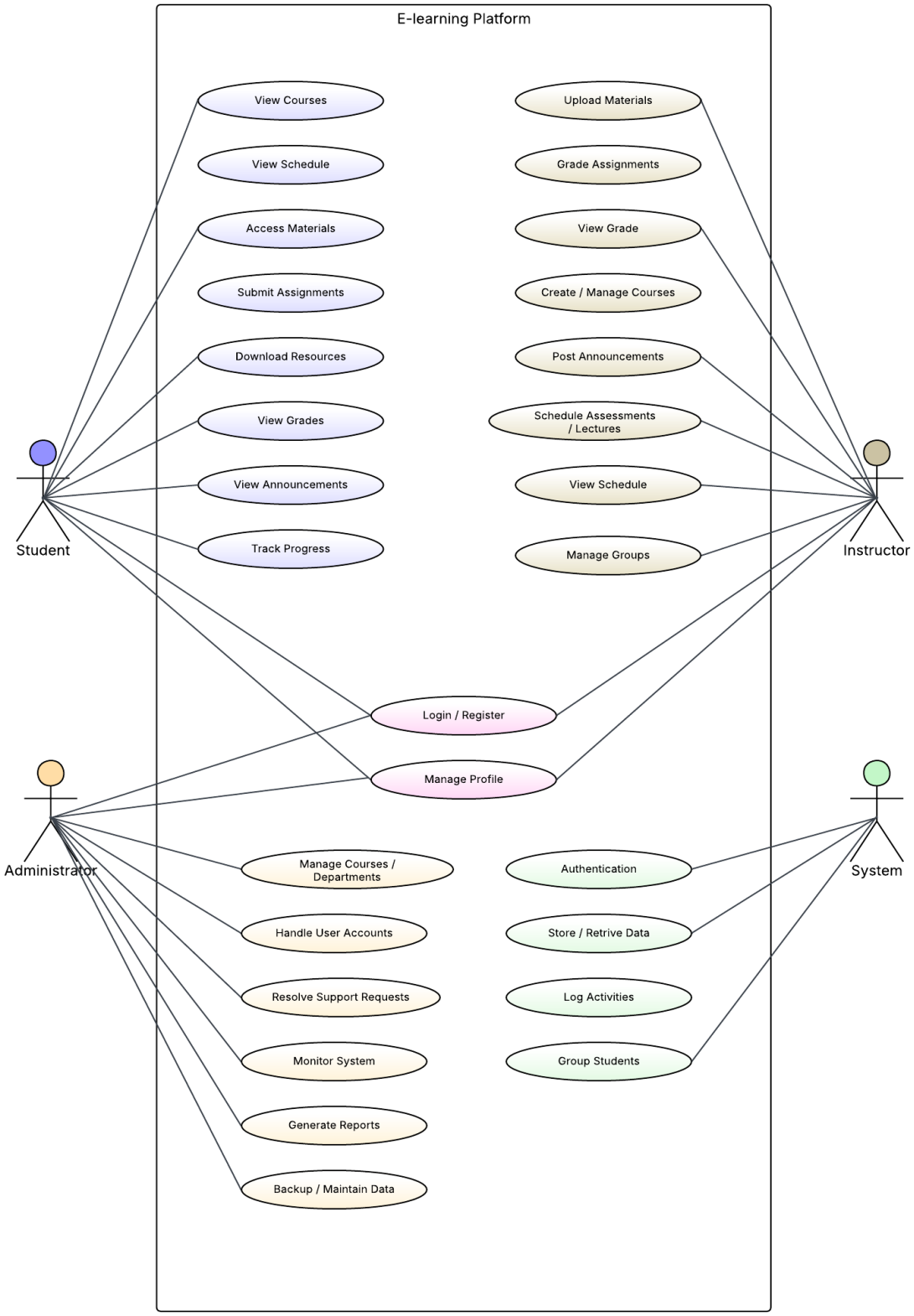

Figure 2.1: Use Case Diagram

2.3.2.1 Use Case Relationships

- «include» Relationships

  - ✓ **Manage Profile** «include» **Login / Register**
  - ✓ **View Courses** «include» **Login / Register**
  - ✓ **View Schedule** «include» **Login / Register**
  - ✓ **Access Materials** «include» **View Courses**
  - ✓ **Download Resources** «include» **Access Materials**
  - ✓ **Submit Assignments** «include» **Access Materials**
  - ✓ **View Grades** «include» **Login / Register**
  - ✓ **Track Progress** «include» **View Grades**
  - ✓ **Upload Materials** «include» **Create / Manage Courses**
  - ✓ **Grade Assignments** «include» **Submit Assignments**
  - ✓ **Manage Groups** «include» **Group Students**
  - ✓ **Handle User Accounts** «include» **Authentication**
  - ✓ **Backup / Maintain Data** «include» **Store / Retrieve Data**
  - ✓ **Monitor System** «include» **Log Activities**

- «extend» Relationships

  - ✓ **Register** «extend» **Login**
  - ✓ **Offline Access** «extend» **Access Materials**
  - ✓ **Post Announcements** «extend» **Create / Manage Courses**
  - ✓ **View Announcements** «extend» **Login / Register**
  - ✓ **Resolve Support Requests** «extend» **Monitor System**
  - ✓ **Generate Reports** «extend» **Monitor System**

- System-Level Use Case Relationships

  - ✓ **Authentication is used by:**
    - Login / Register
    - Manage Profile
    - Handle User Accounts
  - ✓ **Store / Retrieve Data supports:**
    - Access Materials
    - Submit Assignments
    - View Grades
    - Generate Reports
  - ✓ **Group Students is triggered by:**
    - Manage Groups
    - Create / Manage Courses
  - ✓ **Log Activities supports:**
    - Monitor System
    - Resolve Support Requests

2.3.2.2 Use Case Specification tables

Table 2.1: Use Case – Login / Register

Item	**Description**
Use Case ID	UC-01
Use Case Name	Login / Register
Actor(s)	Student, Instructor, Administrator
Description	Allows users to create an account or log in to the system using valid credentials.
Preconditions	User has access to the system interface.
Postconditions	User is authenticated and granted access based on role.
Main Flow	1. User opens login/register page 2. User enters credentials or registration details 3. System validates input4. System grants access
Alternative Flow	Invalid credentials → System displays error message

Table 2.2: Use Case – Manage Profile

Item	**Description**
Use Case ID	UC-02
Use Case Name	Manage Profile
Actor(s)	Student, Instructor, Administrator
Description	Allows users to view and update personal information.
Preconditions	User is logged in.
Postconditions	Profile information is updated and saved.
Main Flow	1. User accesses profile section 2. User edits information 3. System validates data 4. System saves changes
Alternative Flow	Invalid input → System prompts correction

Table 2.3: Use Case – View Courses

Item	**Description**
Use Case ID	UC-03
Use Case Name	View Courses
Actor(s)	Student
Description	Allows students to view courses related to their department or enrollment.
Preconditions	Student is logged in.
Postconditions	Course list is displayed.
Main Flow	1. Student selects “View Courses” 2. System retrieves course data 3. Courses are displayed

Table 2.4: Use Case – View Schedule

Item	**Description**
Use Case ID	UC-04
Use Case Name	View Schedule
Actor(s)	Student, Instructor
Description	Displays class schedules, assessments, and deadlines.
Preconditions	User is logged in.
Postconditions	Schedule is displayed.
Main Flow	1. User selects schedule option 2. System retrieves schedule 3. Schedule is displayed

Table 2.5: Use Case – Access Materials

Item	**Description**
Use Case ID	UC-05
Use Case Name	Access Materials
Actor(s)	Student
Description	Allows students to access course learning materials.
Preconditions	Student is logged in and enrolled in course.
Postconditions	Materials are displayed.
Main Flow	1. Student selects course 2. Student selects materials 3. System displays content

Table 2.6: Use Case – Download Resources

Item	**Description**
Use Case ID	UC-06
Use Case Name	Download Resources
Actor(s)	Student
Description	Enables students to download course materials for offline use.
Preconditions	Student has accessed course materials.
Postconditions	Resource is downloaded.
Main Flow	1. Student selects download option 2. System downloads resource

Table 2.7: Use Case – Submit Assignments

Item	**Description**
Use Case ID	UC-07
Use Case Name	Submit Assignments
Actor(s)	Student
Description	Allows students to submit assignments electronically.
Preconditions	Student is logged in and assignment is available.
Postconditions	Assignment is stored and submission confirmed.
Main Flow	1. Student selects assignment 2. Uploads file 3. System validates and confirms submission
Alternative Flow	Late submission → System displays warning

Table 2.8: Use Case – View Grades

Item	**Description**
Use Case ID	UC-08
Use Case Name	View Grades
Actor(s)	Student
Description	Allows students to view grades and feedback.
Preconditions	Student is logged in; grades are published.
Postconditions	Grades are displayed.
Main Flow	1. Student selects grades 2. System retrieves data 3. Grades displayed

Table 2.9: Use Case – View Announcements

Item	**Description**
Use Case ID	UC-09
Use Case Name	View Announcements
Actor(s)	Student
Description	Displays announcements and notifications to students.
Preconditions	Student is logged in.
Postconditions	Announcements are viewed.
Main Flow	1. Student opens announcements 2. System displays messages

Table 2.10: Use Case – Track Progress

Item	**Description**
Use Case ID	UC-10
Use Case Name	Track Progress
Actor(s)	Student
Description	Allows students to monitor academic progress.
Preconditions	Student is logged in.
Postconditions	Progress report is displayed.
Main Flow	1. Student selects progress option 2. System calculates progress 3. Progress is displayed

Table 2.11: Use Case – Upload Materials

Item	**Description**
Use Case ID	UC-11
Use Case Name	Upload Materials
Actor(s)	Instructor
Description	Allows instructors to upload learning materials.
Preconditions	Instructor is logged in and assigned to course.
Postconditions	Materials are stored and accessible.
Main Flow	1. Instructor selects course 2. Uploads materials 3. System stores materials

Table 2.12: Use Case – Grade Assignments

Item	**Description**
Use Case ID	UC-12
Use Case Name	Grade Assignments
Actor(s)	Instructor
Description	Allows instructors to grade submitted assignments.
Preconditions	Assignments are submitted.
Postconditions	Grades are saved and published.
Main Flow	1. Instructor selects assignment 2. Enters grades 3. System saves grades

Table 2.13: Use Case – Create / Manage Courses

Item	**Description**
Use Case ID	UC-13
Use Case Name	Create / Manage Courses
Actor(s)	Instructor
Description	Enables instructors to create and manage courses.
Preconditions	Instructor is logged in.
Postconditions	Course details are updated.
Main Flow	1. Instructor creates/edits course 2. System saves data

Table 2.14: Use Case – Post Announcements

Item	**Description**
Use Case ID	UC-14
Use Case Name	Post Announcements
Actor(s)	Instructor, Administrator
Description	Allows posting of announcements to students.
Preconditions	User is logged in.
Postconditions	Announcement is published.
Main Flow	1. User creates announcement 2. System publishes

Table 2.15: Use Case – Manage Groups

Item	**Description**
Use Case ID	UC-15
Use Case Name	Manage Groups
Actor(s)	Instructor
Description	Allows instructors to manage student groups.
Preconditions	Course exists.
Postconditions	Groups are updated.
Main Flow	1. Instructor views groups 2. Modifies groups 3. System saves changes

Table 2.16: Use Case – Manage Courses / Departments

Item	**Description**
Use Case ID	UC-16
Use Case Name	Manage Courses / Departments
Actor(s)	Administrator
Description	Allows administrators to manage academic structures.
Preconditions	Administrator is logged in.
Postconditions	Data is updated.
Main Flow	1. Admin edits data 2. System saves

Table 2.17: Use Case – Handle User Accounts

Item	**Description**
Use Case ID	UC-17
Use Case Name	Handle User Accounts
Actor(s)	Administrator
Description	Allows management of user accounts.
Preconditions	Administrator is logged in.
Postconditions	User account status updated.
Main Flow	1. Admin selects user 2. Modifies account 3. System saves

Table 2.18: Use Case – Resolve Support Requests

Item	**Description**
Use Case ID	UC-18
Use Case Name	Resolve Support Requests
Actor(s)	Administrator
Description	Handles system support issues.
Preconditions	Support request exists.
Postconditions	Issue is resolved and logged.
Main Flow	1. Admin reviews request 2. Resolves issue 3. System logs action

Table 2.19: Use Case – Monitor System

Item	**Description**
Use Case ID	UC-19
Use Case Name	Monitor System
Actor(s)	Administrator
Description	Monitors system activity and performance.
Preconditions	Administrator is logged in.
Postconditions	System status displayed.
Main Flow	1. Admin accesses dashboard 2. System displays metrics

Table 2.20: Use Case – Generate Reports

Item	**Description**
Use Case ID	UC-20
Use Case Name	Generate Reports
Actor(s)	Administrator
Description	Generates academic and system reports.
Preconditions	Data exists.
Postconditions	Report is generated.
Main Flow	1. Admin selects report 2. System generates output

Table 2.21: Use Case – Backup / Maintain Data

Item	Description
Use Case ID	UC-21
Use Case Name	Backup / Maintain Data
Actor(s)	Administrator
Description	Performs data backup and maintenance operations.
Preconditions	System is operational.
Postconditions	Data is backed up successfully.
Main Flow	1. Admin initiates backup 2. System stores backup

2.3.3 Business Rule Documentation

BR-01 – User Authentication

- **Description:** All users (students, instructors, administrators) must be authenticated before accessing the system.
- **Purpose:** Ensures system security and protects sensitive data.

BR-02 – Role-Based Access

- **Description:** Users can only access features permitted by their role (Student, Instructor, Administrator).
- **Purpose:** Maintains data integrity and prevents unauthorized access.

BR-03 – Course Enrollment Restriction

- **Description:** Students can only view or enroll in courses available to their major/department.
- **Purpose:** Ensures relevance and avoids scheduling conflicts.

BR-04 – Assignment Submission Deadline

- **Description:** Students must submit assignments before the specified deadline; late submissions are flagged.
- **Purpose:** Enforces academic policies and fairness.

BR-05 – Group Assignment

- **Description:** The system automatically assigns students to project or practical groups based on predefined rules (e.g., class, major, skill level).
- **Purpose:** Simplifies group formation and ensures balanced teams.

BR-06 – Data Validation

- **Description:** All input data (registration, assignment uploads, grades) must be validated for correctness and completeness.
- **Purpose:** Prevents errors and maintains data integrity.

BR-07 – Grade Confidentiality

- **Description:** Only instructors and relevant administrators can view or edit grades; students see only their own grades.
- **Purpose:** Ensures privacy and compliance with academic regulations.

BR-08 – Material Accessibility

- **Description:** Only enrolled students can access course materials; materials must be available in multiple formats when possible (PDF, video, slides).
- **Purpose:** Supports student learning while securing content.

BR-09 – Announcement Visibility

- **Description:** Announcements are visible only to the target audience (course students or all users) as specified by the poster.
- **Purpose:** Prevents irrelevant information from being displayed to users.

BR-10 – Offline Access

- **Description:** Students may download course materials for offline use; synchronization occurs automatically when online.
- **Purpose:** Supports flexible learning environments.

BR-11 – Data Backup

- **Description:** All academic and user data must be backed up periodically according to institutional policy.
- **Purpose:** Prevents data loss and ensures system reliability.

BR-12 – System Logging

- **Description:** The system must log all user activities (login, uploads, submissions, grades) for auditing and troubleshooting purposes.
- **Purpose:** Supports accountability, monitoring, and troubleshooting.

BR-13 – Report Generation

- **Description:** Administrators can generate reports on course performance, system usage, and student progress.
- **Purpose:** Supports decision-making and academic planning.

BR-14 – Security Compliance

- **Description:** The system must comply with data protection and privacy regulations applicable to educational institutions.
- **Purpose:** Ensures legal compliance and user trust.

BR-16 – System Availability

- **Description:** The LMS should be available 24/7 with minimal downtime; maintenance should be scheduled and communicated.
- **Purpose:** Ensures continuous access to learning resources.

BR-17 – Notification Delivery

- **Description:** Notifications and announcements must be delivered in real-time to the target users.
- **Purpose:** Improves communication efficiency.

BR-18 – Assignment Feedback

- **Description:** Instructors must provide feedback for graded assignments within a defined timeframe.
- **Purpose:** Enhances learning and supports academic policies.

2.3.4 User Interface prototype

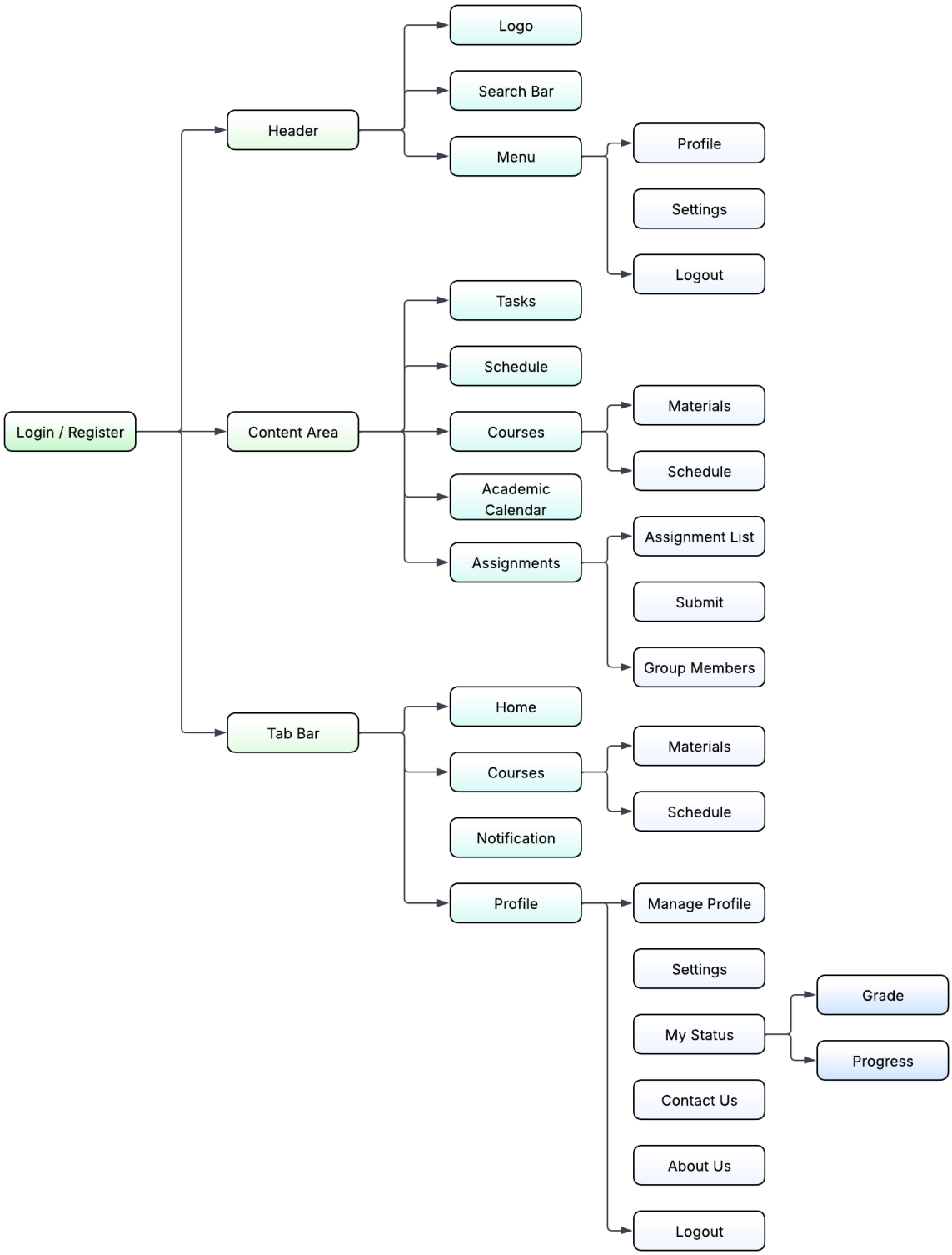

Figure 2.2: Prototype – Student UI

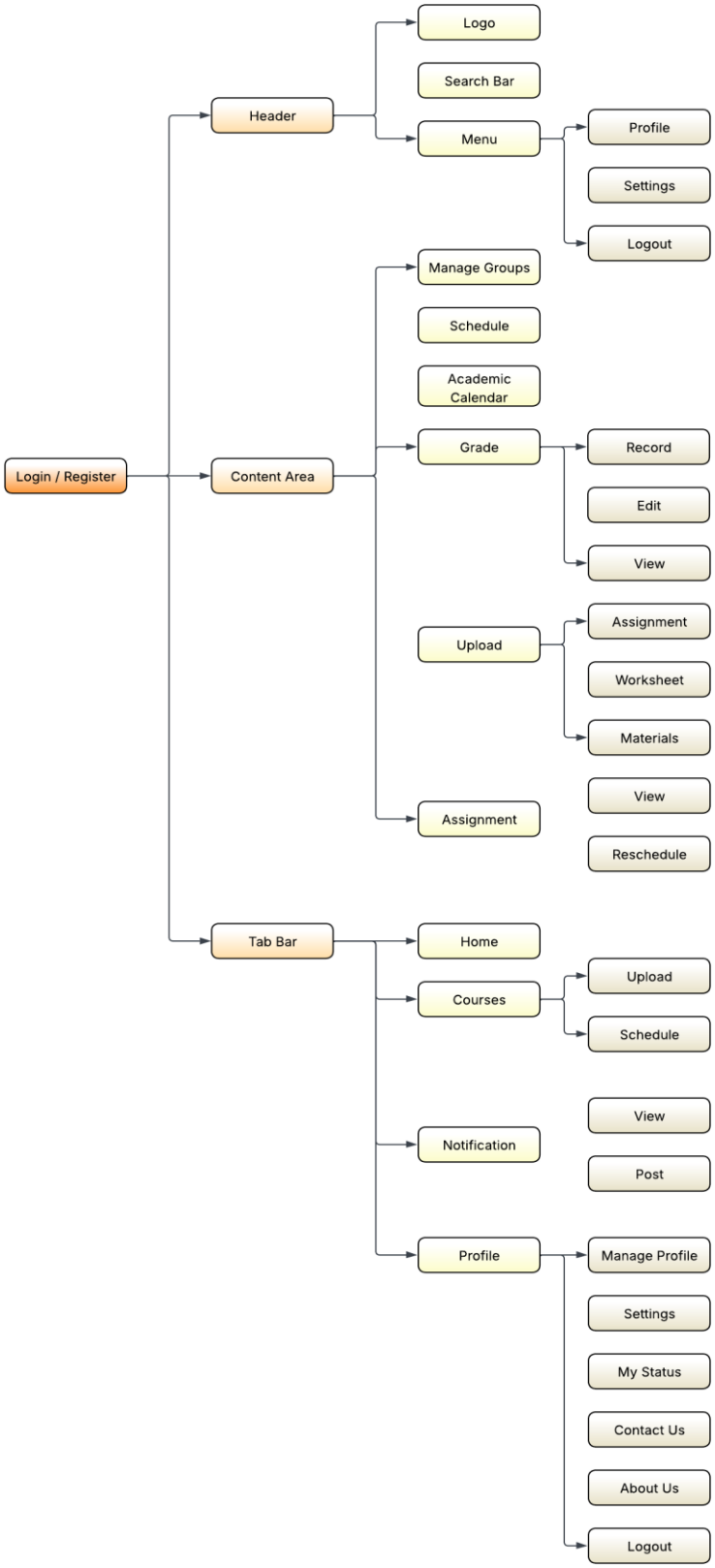

Figure 2.3: Prototype – Instructor UI

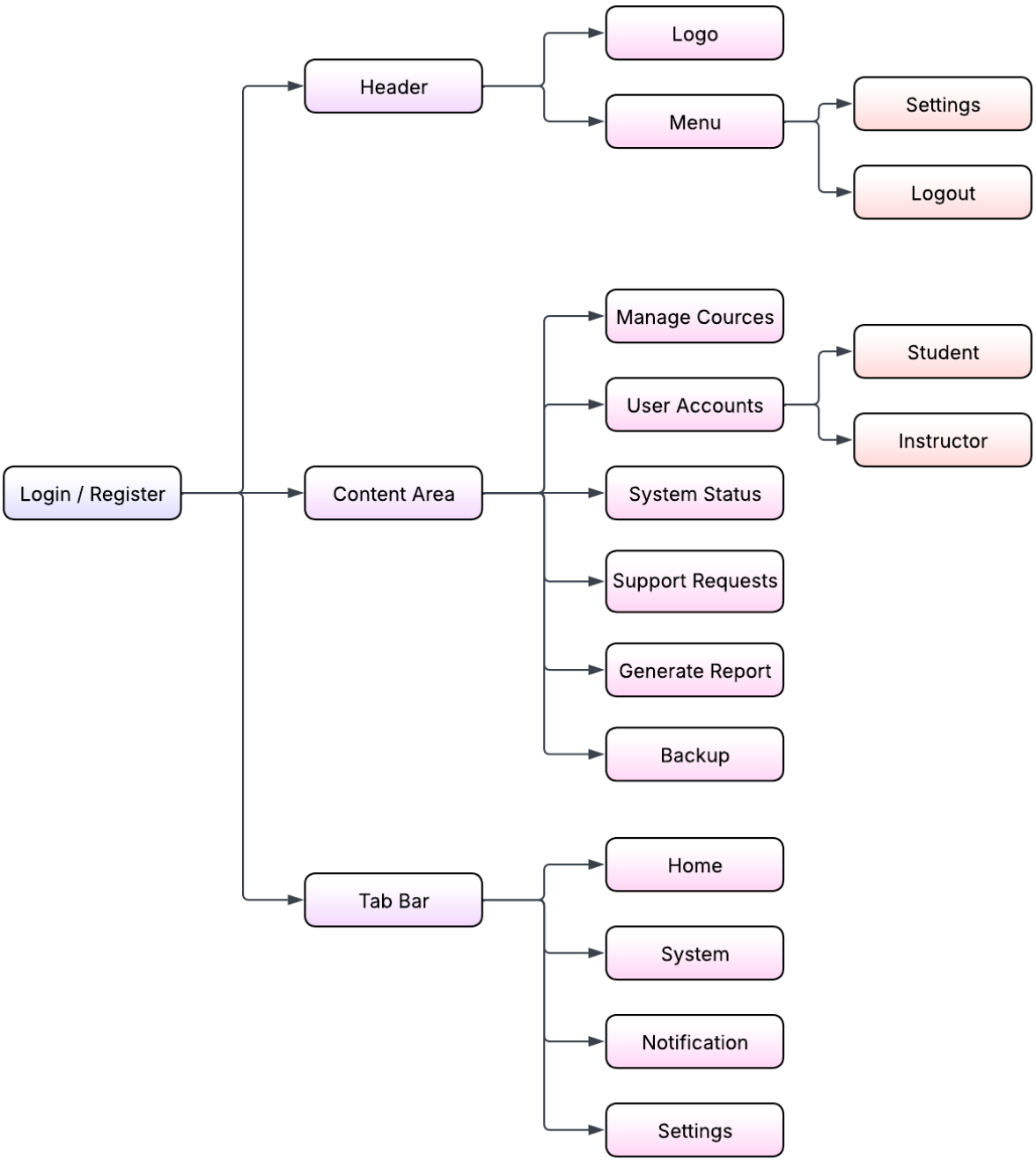

Figure 2.4: Prototype – Admin UI

2.3.5 State chart diagram

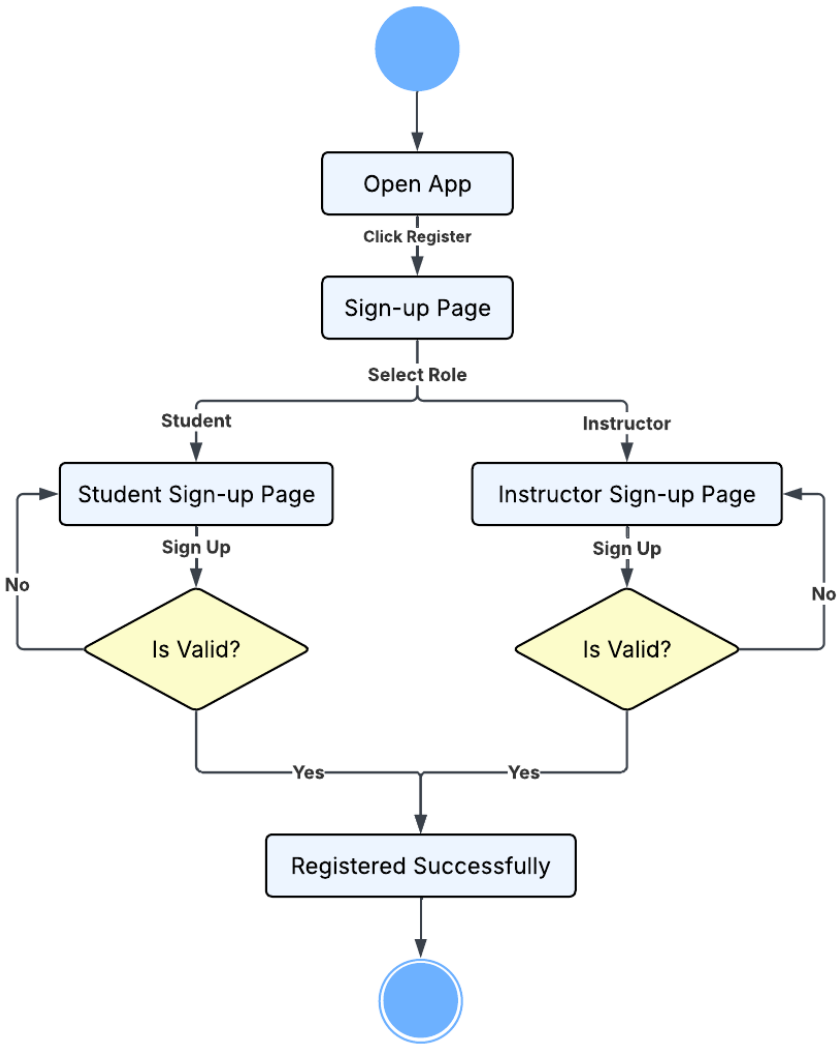

Figure 2.5: State Chart Diagram – Register

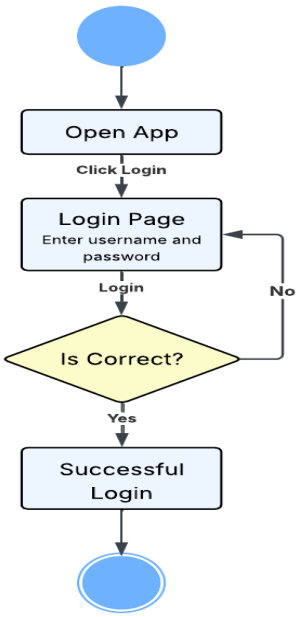

Figure 2.6: State Chart Diagram – Login

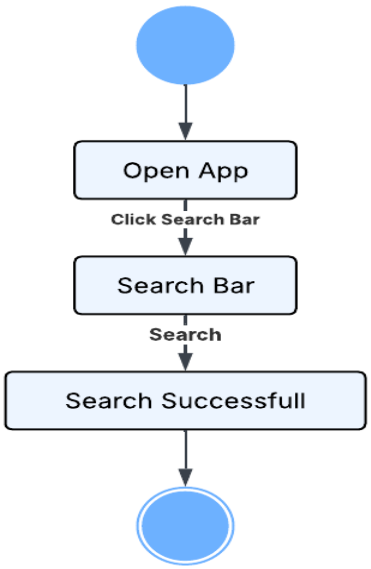

Figure 2.7: State Chart Diagram – Search

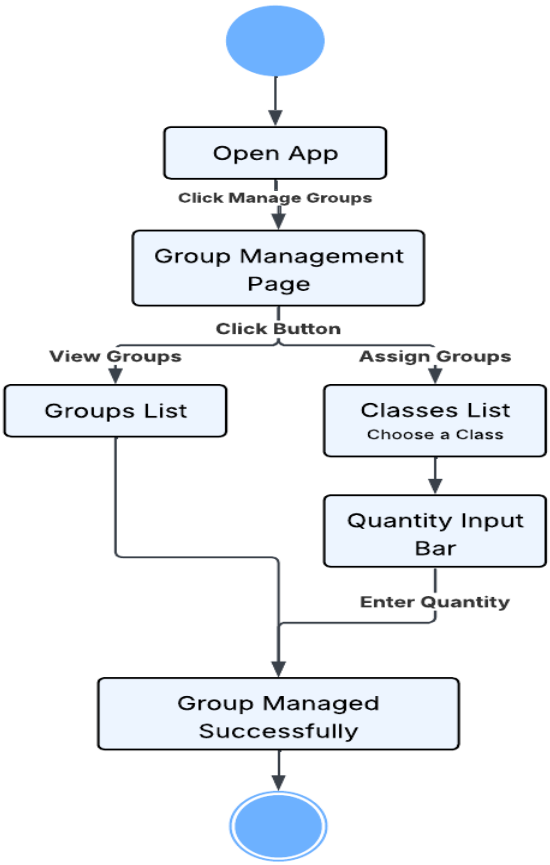

Figure 2.8: State Chart Diagram – Group Management for Instructor

2.3.6 Activity Diagram

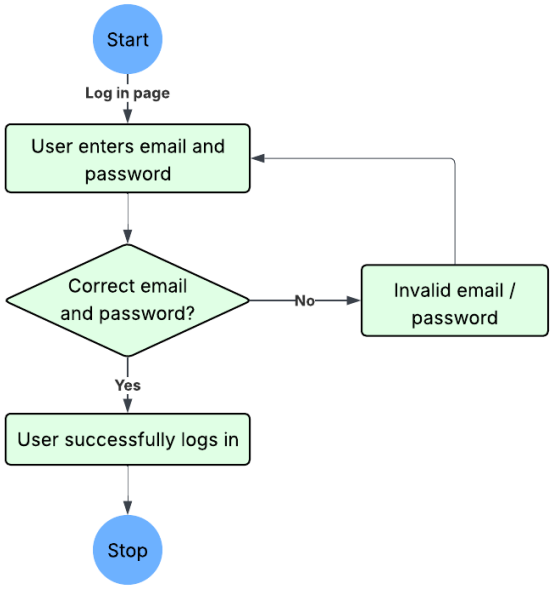

Figure 2.9: Activity Diagram – Login

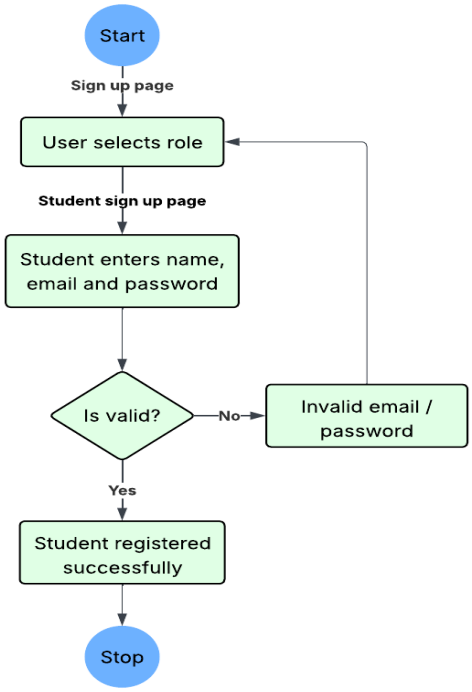

Figure 2.10: Activity Diagram – Register Student

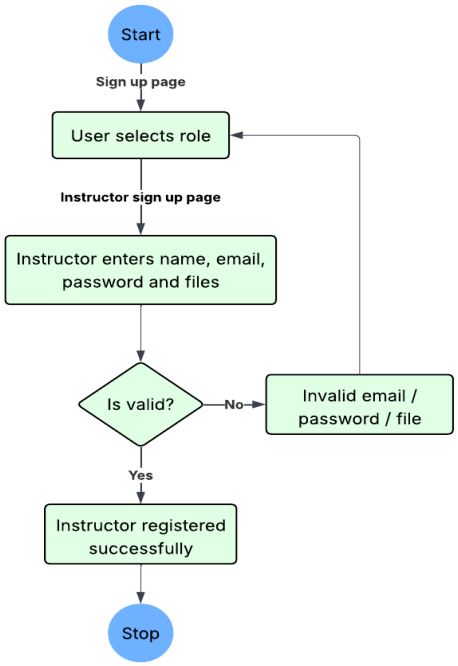

Figure 2.11: Activity Diagram – Register Instructor

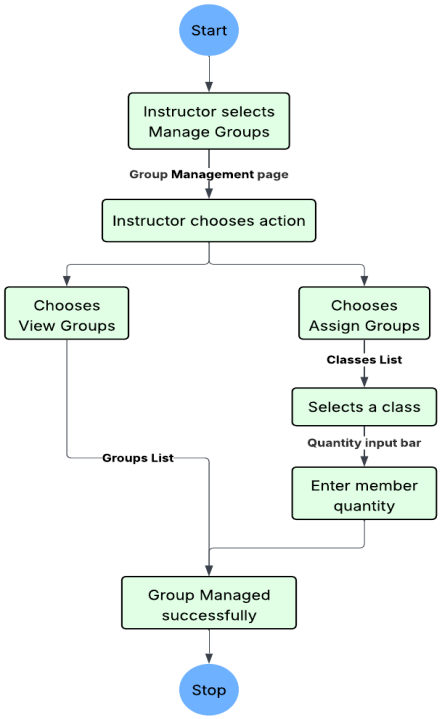

Figure 2.12: Activity Diagram – Group Management for Instructor

2.3.7 Sequence diagram

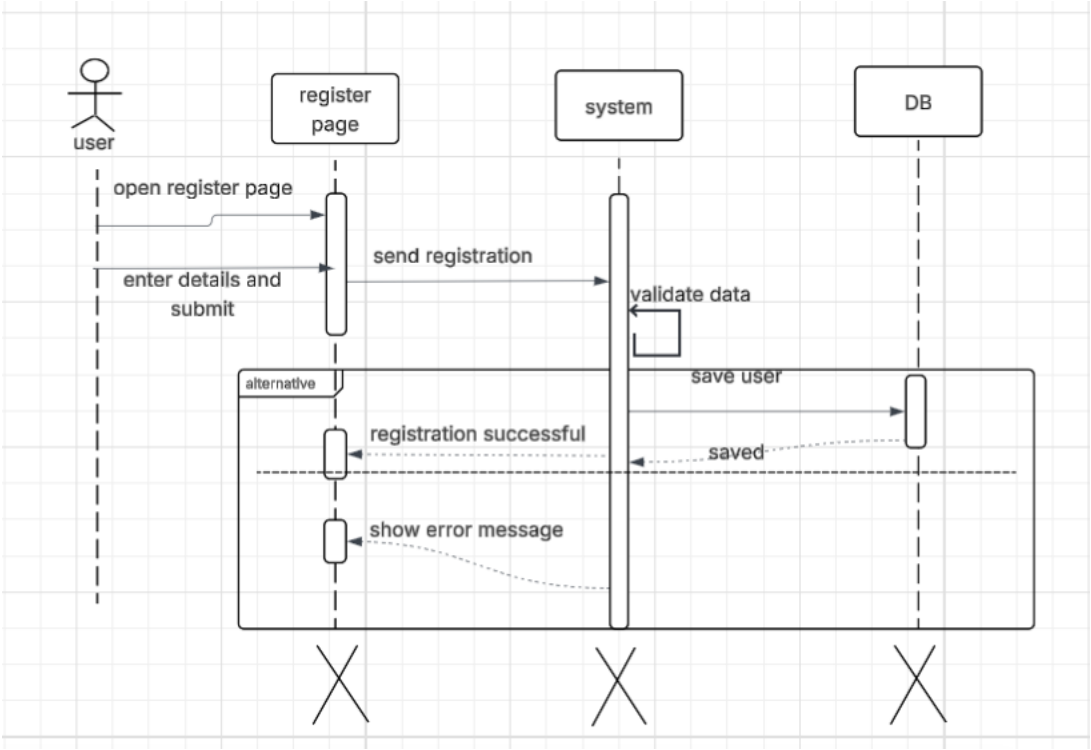

Figure 2.13: Sequence Diagram – Register

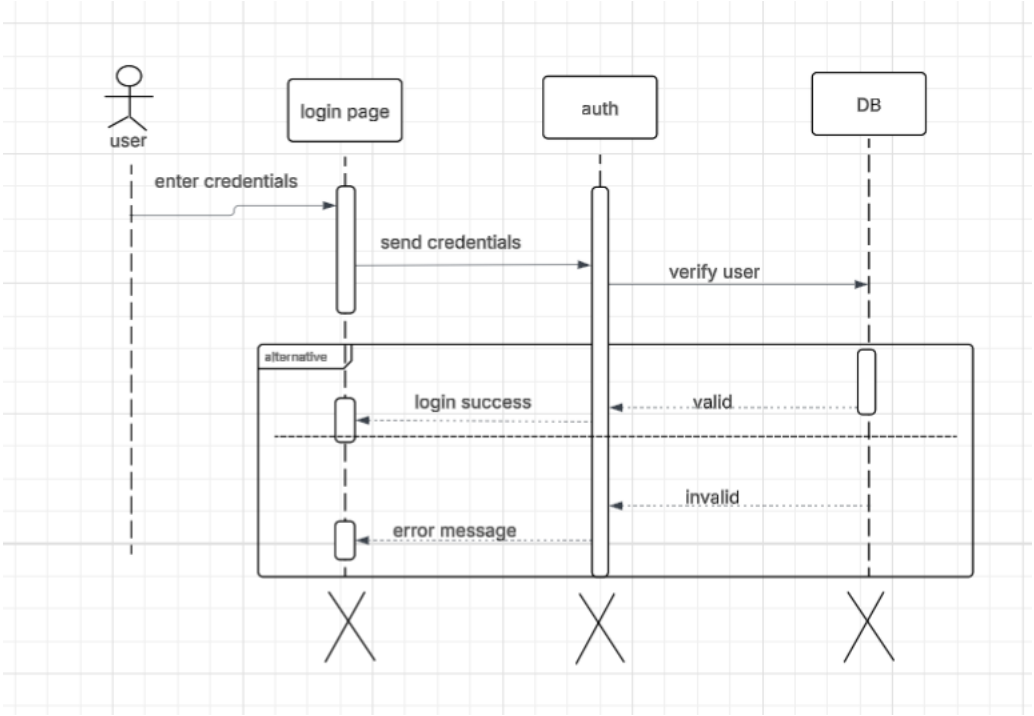

Figure 2.14: Sequence Diagram – Login

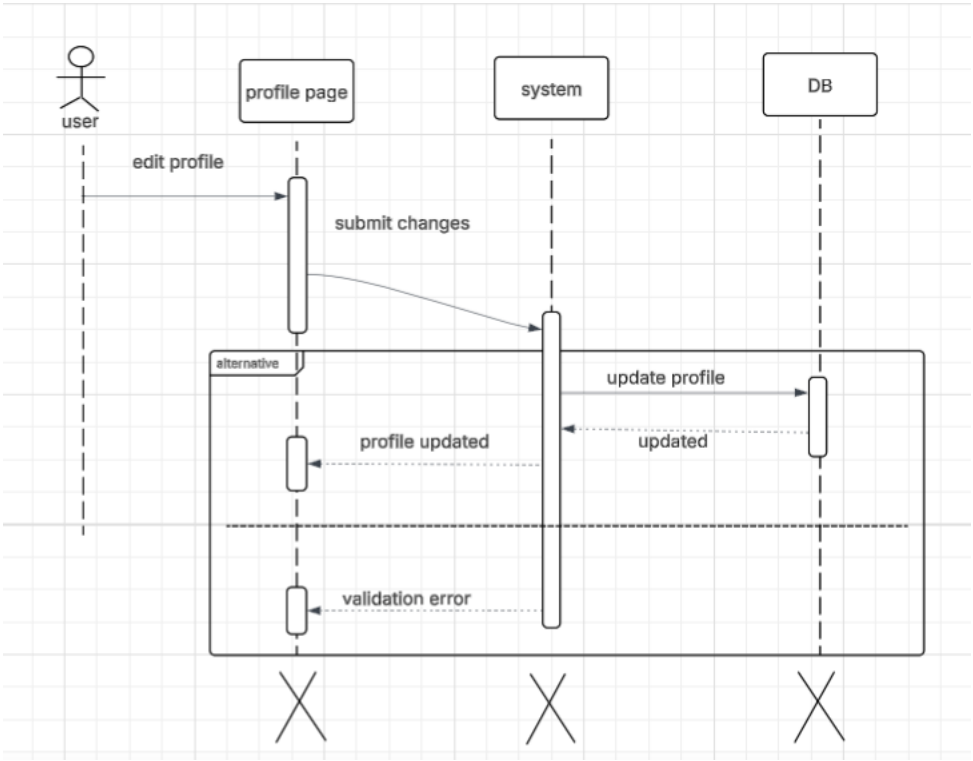

Figure 2.15: Sequence Diagram – Manage Profile

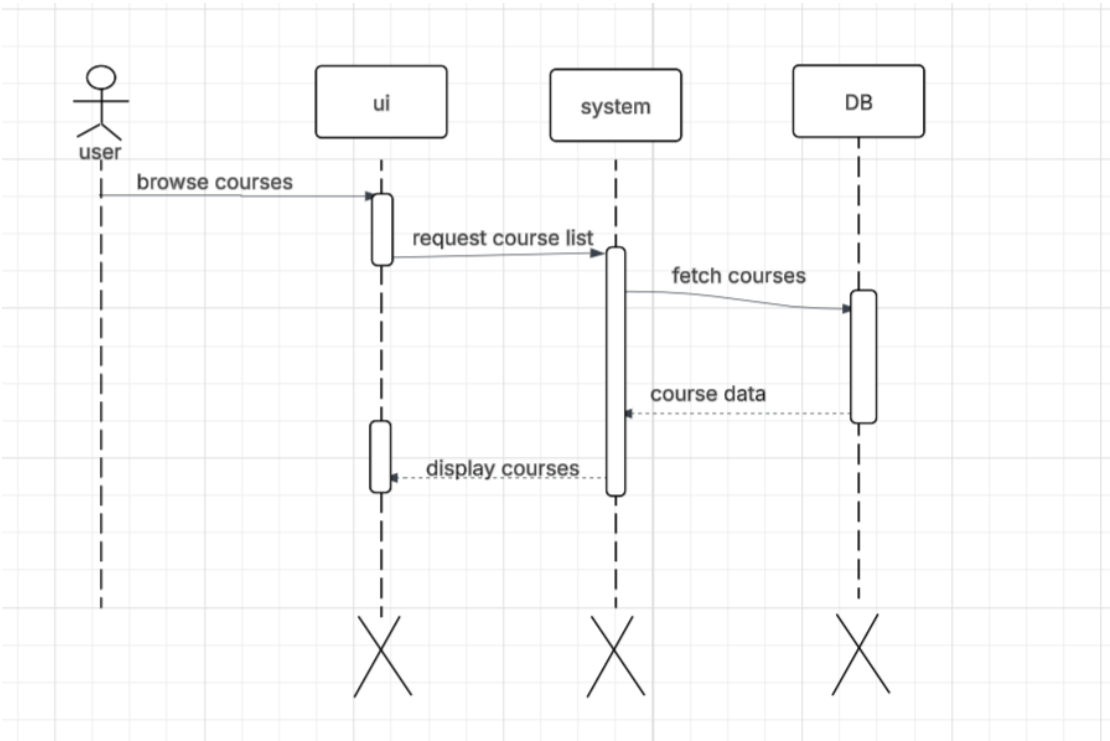

Figure 2.16: Sequence Diagram – Browse Courses

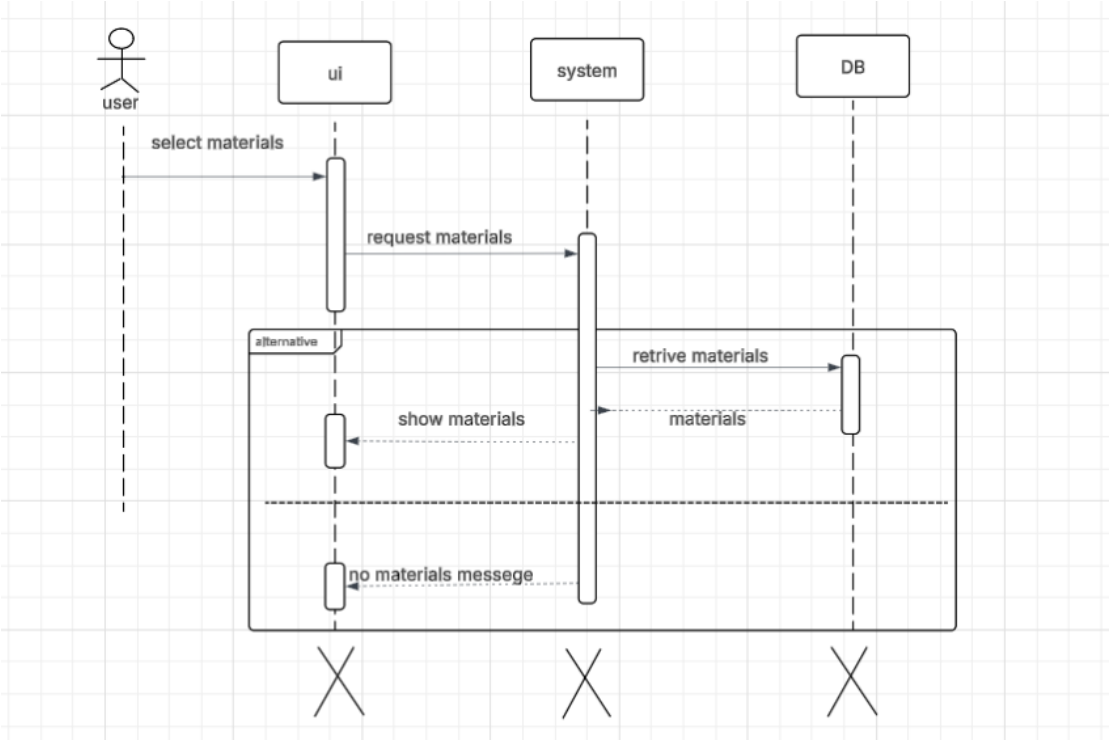

Figure 2.17: Sequence Diagram – Access Material

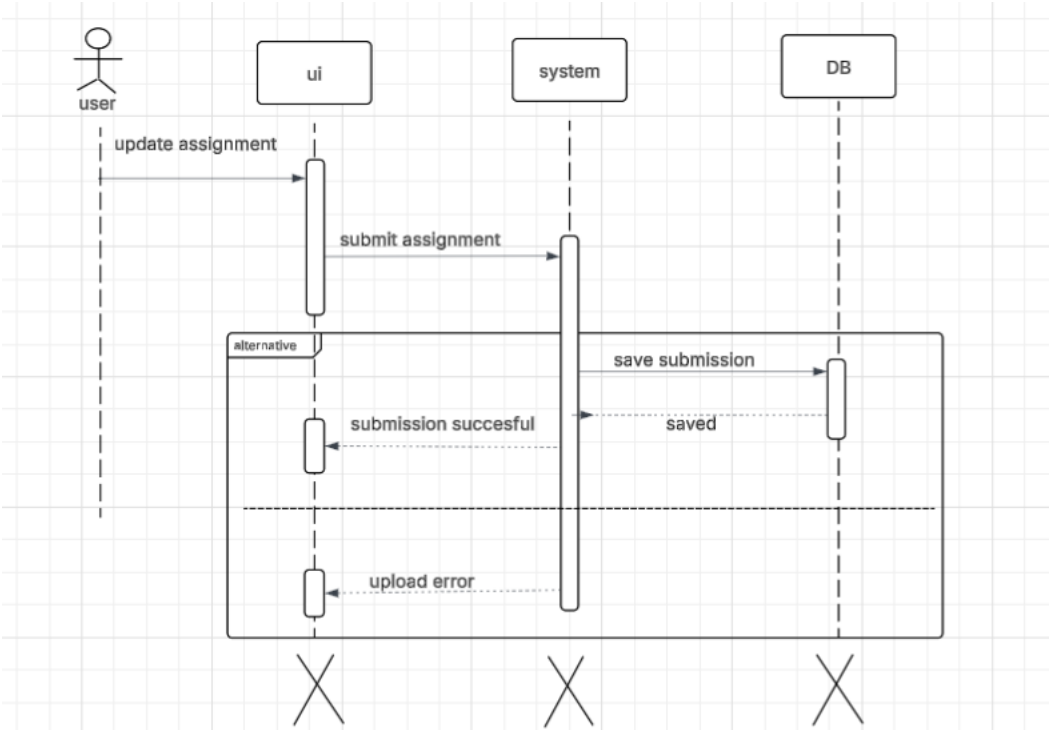

Figure 2.18: Sequence Diagram – Submit Assignment

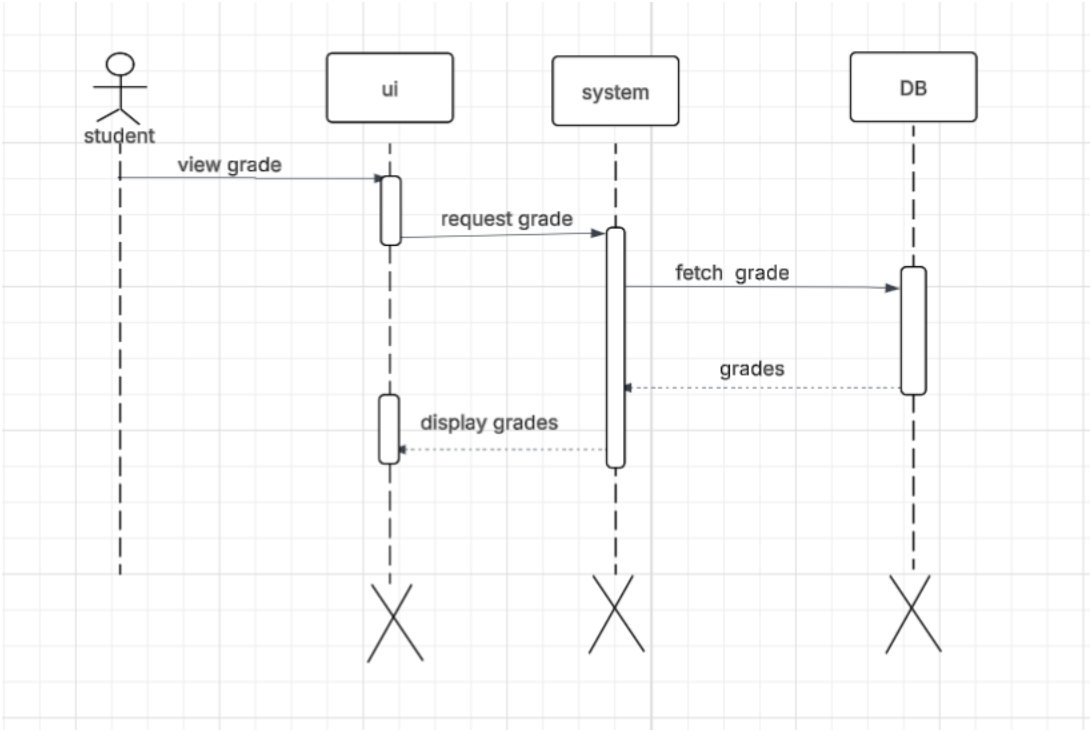

Figure 2.19: Sequence Diagram – View Grade

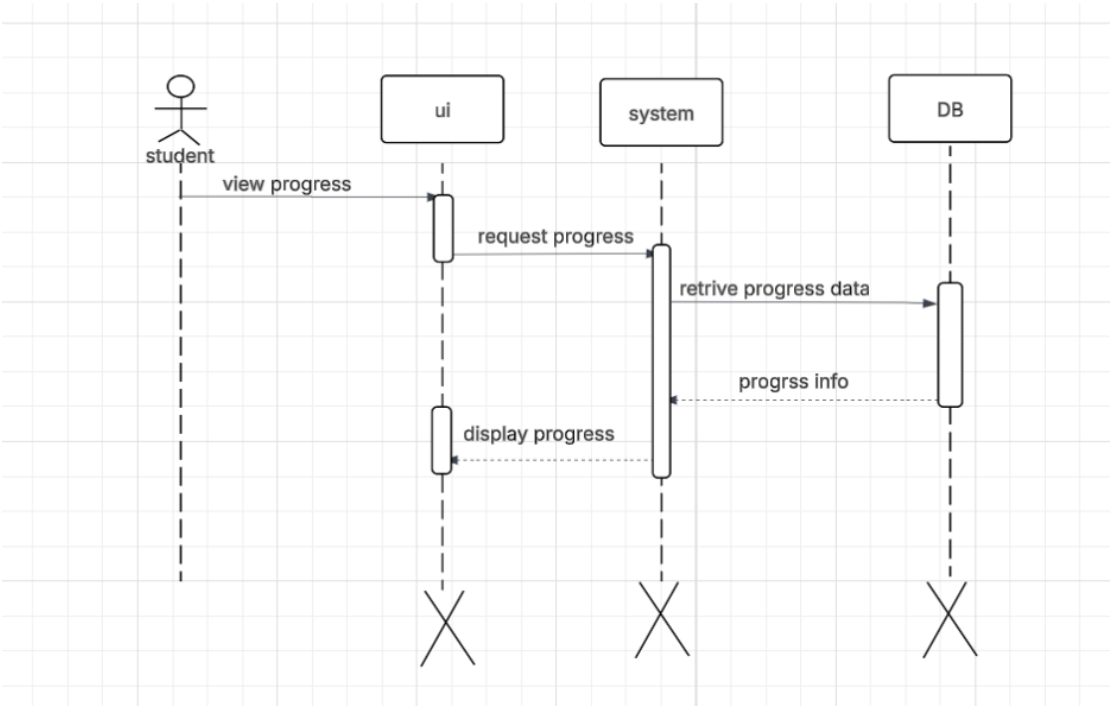

Figure 2.20: Sequence Diagram – Track Progress

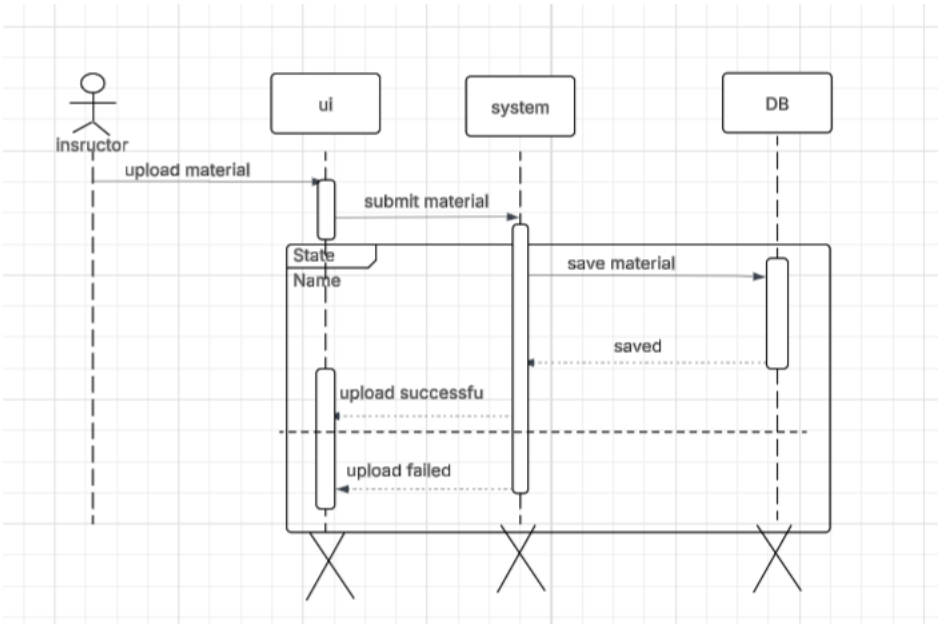

Figure 2.21: Sequence Diagram – Upload Material

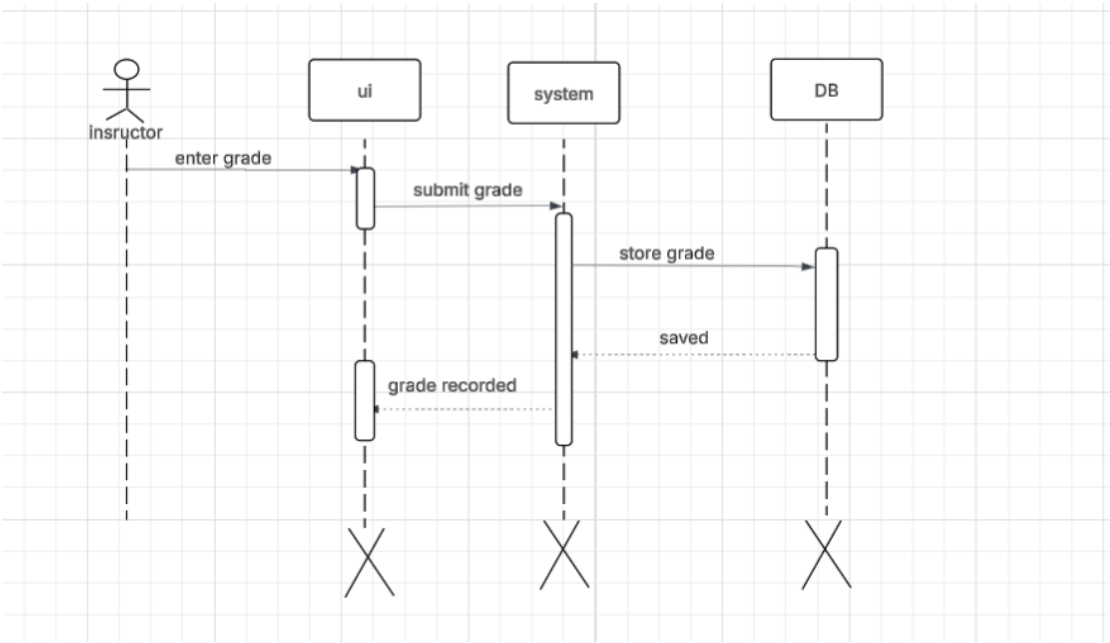

Figure 2.22: Sequence Diagram – Grade Assignment

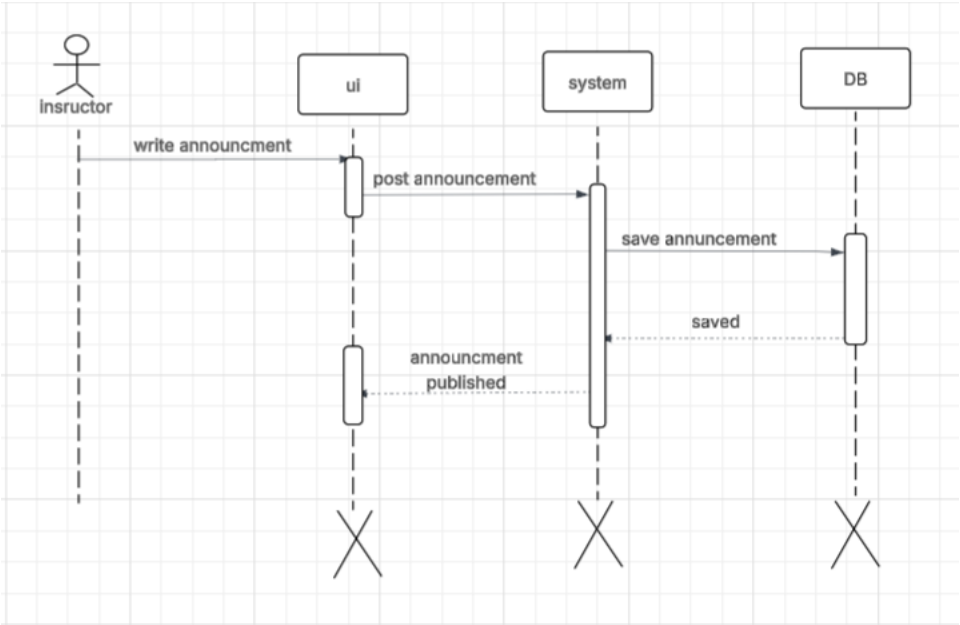

Figure 2.23: Sequence Diagram – Send Announcement

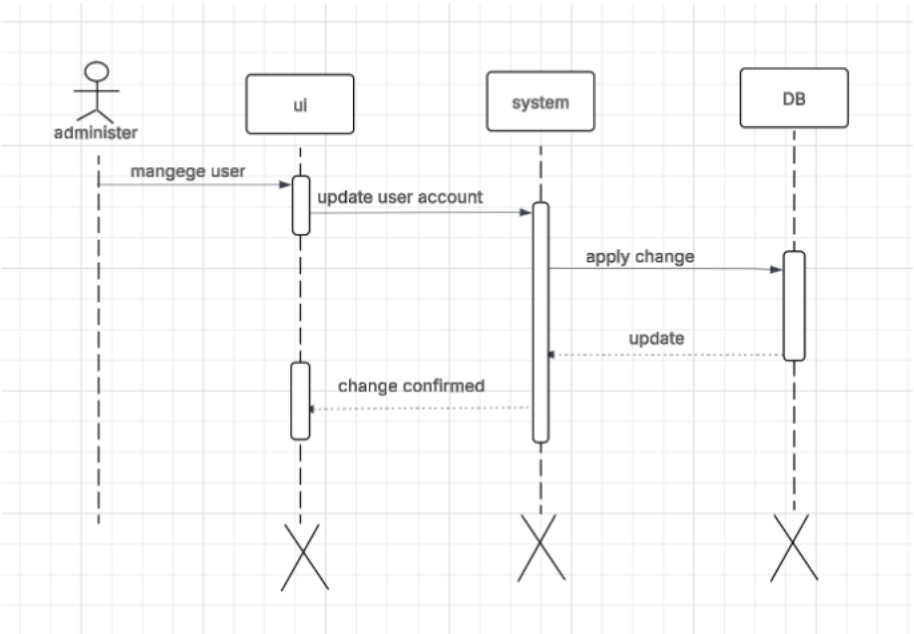

Figure 2.24: Sequence Diagram – Handle User Account

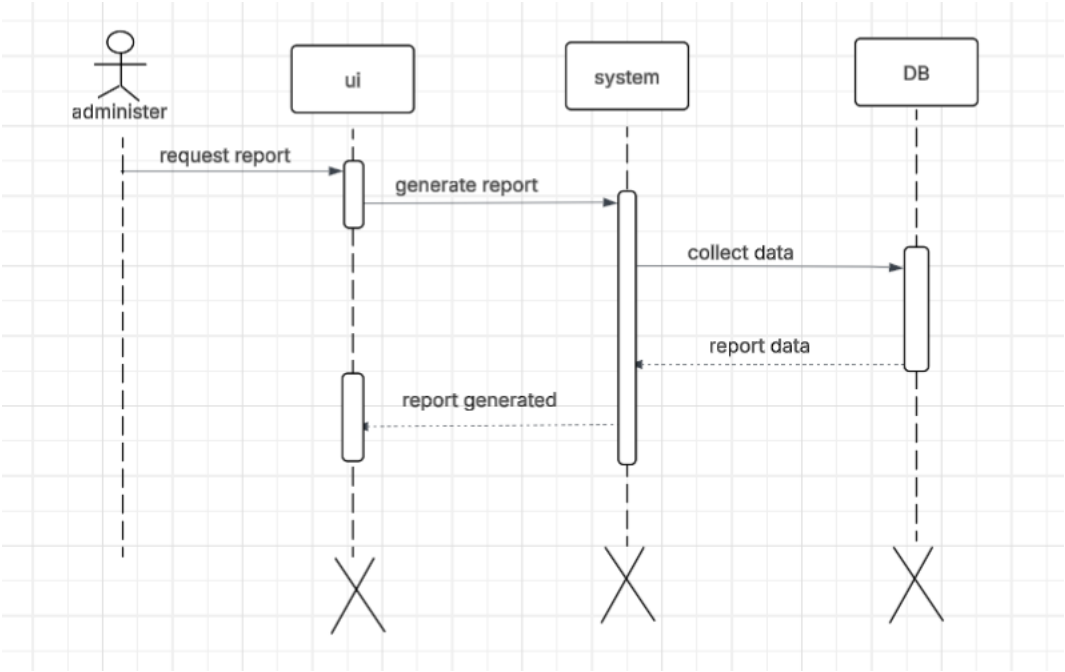

Figure 2.25: Sequence Diagram – Generate Reports

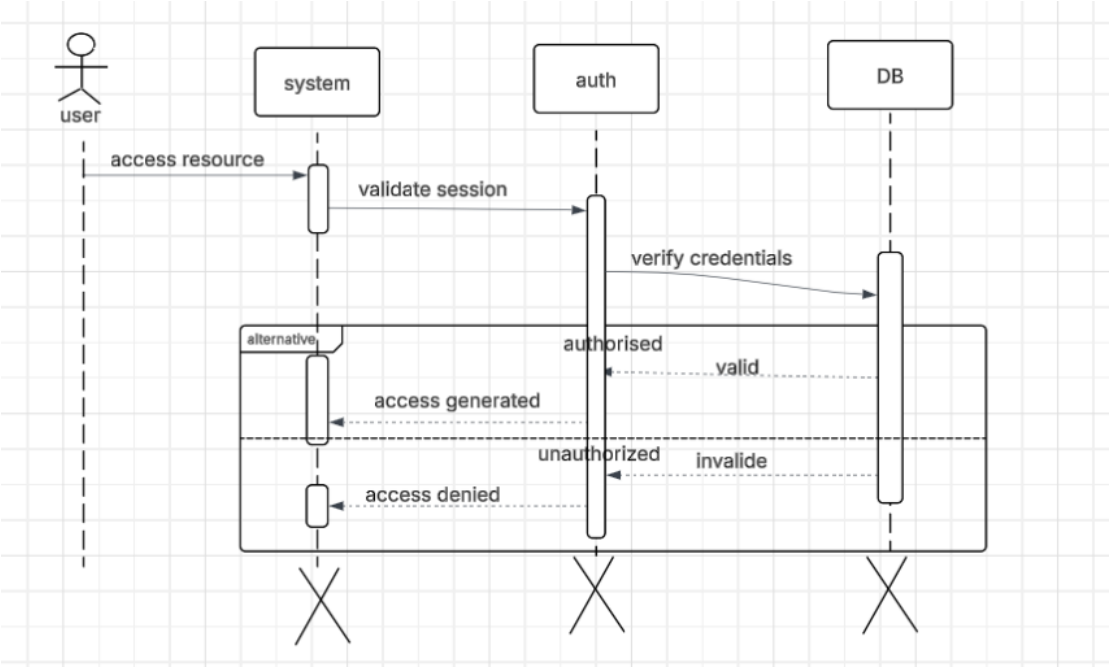

Figure 2.26: Sequence Diagram – Authentication

2.3.8 Analysis Class Model

A. Classes, Attributes, and Operations

1. User

Attributes:

- userID: String
- name: String
- email: String
- password: String
- role: Enum (Student, Instructor, Administrator)
- profileDetails: Profile

Operations:

- login()
- logout()
- updateProfile(profileDetails)
- recoverPassword()

Notes: Superclass for Student, Instructor, and Administrator.

2. Student (inherits User)

Attributes:

- studentID: String
- major: String
- enrolledCourses: List
- progress: ProgressReport

Operations:

- viewCourses()
- viewSchedule()
- accessMaterials(courseID)
- downloadResource(resourceID)
- submitAssignment(assignmentID, file)
- viewGrades()
- trackProgress()

3. Instructor (inherits User)

Attributes:

- instructorID: String

- coursesTaught: List
- officeHours: Schedule

Operations:

- createCourse(course: Course)
- manageCourse(courseID)
- uploadMaterial(courseID, material: Material)
- gradeAssignment(assignmentID, grade)
- postAnnouncement(announcement: Announcement)
- scheduleAssessment(assessment: Assessment)
- manageGroups(courseID, group: Group)

4. Administrator (inherits User)

Attributes:

- adminID: String
- managedDepartments: List

Operations:

- handleUserAccounts(user: User)
- manageCoursesDepartment(department: Department)
- resolveSupportRequest(request: SupportRequest)
- monitorSystem()
- generateReports(reportType: String)
- backupData()

5. Course

Attributes:

- courseID: String
- courseName: String
- department: Department
- instructor: Instructor
- enrolledStudents: List
- materials: List
- schedule: Schedule
- groups: List

Operations:

- addMaterial(material: Material)
- removeMaterial(materialID)
- enrollStudent(studentID)

- assignGroup(group: Group)

6. Material

Attributes:

- materialID: String
- type: Enum (PDF, Video, Slide, Assignment)
- content: File/URL
- courseID: String
- uploadDate: Date

Operations:

- download()
- view()

7. Assignment (inherits Material)

Attributes:

- assignmentID: String
- dueDate: Date
- submittedBy: List

Operations:

- submit(student: Student, file: File)
- grade(instructor: Instructor, grade: Float)

8. Submission

Attributes:

- submissionID: String
- student: Student
- assignment: Assignment
- file: File
- submissionDate: Date
- grade: Float

Operations:

- viewSubmission()
- updateGrade(grade: Float)

9. Group

Attributes:

- groupID: String
- groupName: String
- members: List
- projectAssignments: List

Operations:

- addMember(student: Student)
- removeMember(studentID: String)

10. Department

Attributes:

- departmentID: String
- name: String
- courses: List

Operations:

- addCourse(course: Course)
- removeCourse(courseID: String)

11. Announcement

Attributes:

- announcementID: String
- title: String
- message: String
- postedBy: User
- targetAudience: List
- postDate: Date

Operations:

- post()
- view()

12. Schedule

Attributes:

- scheduleID: String
- courseID: String
- date: Date

- time: Time
- type: Enum (Lecture, Assessment, Event)

Operations:

- viewSchedule()
- updateSchedule()

13. SupportRequest

Attributes:

- requestID: String
- raisedBy: User
- issueDescription: String
- status: Enum (Open, In Progress, Resolved)
- creationDate: Date

Operations:

- submitRequest()
- updateStatus(status: Enum)
- resolveRequest()

Relationships

1. **Inheritance:** User → Student, Instructor, Administrator
2. **Association:**
    - Student ↔ Course (enrolledCourses)
    - Instructor ↔ Course (coursesTaught)
    - Course ↔ Material (materials)
    - Student ↔ Submission (submitted assignments)
    - Group ↔ Student (members)
    - Administrator ↔ Department (managedDepartments)
3. **Composition:**
    - Course → Group (a course has groups)
    - Assignment → Submission (assignment has submissions)
    - Department → Course (department owns courses)
4. **Dependency / Uses:**
    - Announcement used by Course, Instructor, Administrator
    - Schedule used by Course, Student, Instructor

2.3.9 Logic model

1. Login

```
Begin
Load Login Page
Get: Email/Username, Password
User enters login credentials
Click Login button
If (Form is filled)
   If (Valid input values)
      If (User exists in database)
         If (Password matches hashed password)
            Create user session
            Redirect user based on role (Student / Instructor / Administrator)
            Return: Login successful
         Else
            Return: Incorrect password
         ENDIF
      Else
         Return: User not found
      ENDIF
   Else
      Return: Invalid input values
   ENDIF
Else
   Return: Empty submission, please fill all required fields
ENDIF
End
```

2. Manage Profile

```
Begin
Load Profile Page
Get: Updated personal details
User edits profile information
Click Save button
If (User is authenticated)
   If (Form is filled)
      If (Valid input values)
         Update profile in database
         Return: Profile updated successfully
      Else
         Return: Invalid input values
      ENDIF
   Else
      Return: No changes detected
   ENDIF
Else
```

```
    Return: Unauthorized access
  ENDIF
  End
```

3. Logic Model – View Courses

```
  Begin
  Load Courses Page
  If (User is authenticated)
    Retrieve courses based on user role and department
    Display list of courses
  Else
    Redirect to Login Page
  ENDIF
  End
```

4. Access / Download Course Materials

```
  Begin
  Select Course
  Select Material
  If (User is authenticated)
    If (User is enrolled in course)
      Retrieve material from database
      If (Download option selected)
        Save material locally for offline access
      ENDIF
      Display material
    Else
      Return: Access denied
    ENDIF
  Else
    Redirect to Login Page
  ENDIF
  End
```

5. Submit Assignment

```
  Begin
  Load Assignment Submission Page
  Get: Assignment File
  Click Submit button
  If (User is authenticated)
    If (File is uploaded)
      If (Submission deadline not passed)
        Save file to database
        Generate submission confirmation
        Return: Assignment submitted successfully
      Else
```

```
        Return: Submission deadline passed
      ENDIF
    Else
      Return: No file uploaded
    ENDIF
  Else
    Return: Unauthorized access
  ENDIF
  End
```

6. Grade Assignment

```
  Begin
  Load Submitted Assignments
  Select Assignment and Student
  Get: Grade, Feedback
  Click Submit Grade
  If (Instructor is authenticated)
    If (Grade is valid)
      Save grade and feedback to database
      Notify student
      Return: Grade submitted successfully
    Else
      Return: Invalid grade value
    ENDIF
  Else
    Return: Unauthorized access
  ENDIF
  End
```

7. View Grades

```
  Begin
  Load Grades Page
  If (Student is authenticated)
    Retrieve grades from database
    Display grades and feedback
  Else
    Redirect to Login Page
  ENDIF
  End
```

8. Post Announcement

```
  Begin
  Load Announcement Page
  Get: Title, Message, Target Audience
  Click Post button
```

```
If (User is authenticated AND role is Instructor or Administrator)
   If (Title and Message are filled)
      Save announcement to database
      Notify target users
      Return: Announcement posted successfully
   Else
      Return: Missing announcement details
   ENDIF
Else
   Return: Unauthorized access
ENDIF
End
```

9. Automatic Group Assignment

```
Begin
Load Group Management Page
Select Course
Click Generate Groups
If (Instructor is authenticated)
   Retrieve enrolled students
   Apply grouping rules
   Create groups
   Save groups to database
   Return: Groups created successfully
Else
   Return: Unauthorized access
ENDIF
End
```

10. Backup / Maintain Data

```
Begin
Load System Maintenance Panel
Click Backup Data
If (Administrator is authenticated)
   Initiate backup process
   Store backup securely
   Log backup activity
   Return: Backup completed successfully
Else
   Return: Unauthorized access
ENDIF
End
```

Chapter 3: System Design

This chapter focuses on the system design of the E-Learning Management System (ELMS), translating the requirements specified in the previous chapter (Requirements Specification) into a detailed blueprint for the development of the system. The design phase involves creating both high-level and low-level designs, considering various aspects such as architecture, data structures, interfaces, modules, and algorithms.

Regarding the ELMS, a platform connecting students with instructors and administrators, the system design encompasses architectural frameworks, robust local database design, and a user-friendly mobile interface. Security measures, error handling, and performance optimization are integral components, ensuring reliability and scalability within the university's local network. This documentation serves as a guideline for the development team, shaping features that streamline the educational and resource management process on the ELMS platform.

Design Goals

Design goals outline the qualities developers should optimize to meet non-functional requirements (NFRs). These goals, derived from the NFRs, define the system's optimized qualities. The major quality perspectives guiding these objectives include functionality, performance, dependability, reliability, scalability, and maintainability. This framework ensures that the system not only delivers the intended features but also operates efficiently, reliably, and with ease of maintenance, aligning with comprehensive non-functional criteria.

Functionality

The system is designed to fulfill all specified requirements by providing robust features and capabilities tailored to the university environment. This emphasis on functionality aims to provide users with a rich and satisfying experience through core modules such as automated student grouping and digital resource distribution. By automating these processes, the system effectively addresses the diverse needs within the academic environment and streamlines the entire e-learning process for students, instructors, and administrators alike

Performance

Achieving optimal performance is crucial for the success of the ELMS, requiring swift response times and seamless throughput. As a mobile-based platform designed to connect students with instructors, it necessitates fast responses from the backend server to user requests. The system is engineered to efficiently handle multiple concurrent users engaged in high-demand activities, such as accessing and downloading lecture materials in various formats like PDFs and videos, receiving real-time notification deliveries for announcement updates, and managing the digital submission of assignments alongside real-time grade tracking.

Dependability

The platform ensures reliability through a robust local server infrastructure and consistent SQL database management, allowing users to depend on uninterrupted access to academic assistance with minimal downtime. Designed for high availability, it leverages a dependable local network configuration so that educational services remain accessible anytime on campus. Security is a top priority, with a multi-layered defense strategy protecting institutional data. This includes secure SQL query handling, rigorous data validation, robust authentication protocols such as secure login sessions and role-based permissions, and industry-standard encryption for sensitive academic records and user credentials, safeguarding information both in transit and at rest. The system also incorporates effective error-handling mechanisms to maintain stability and user trust, including comprehensive input validation during critical operations like user registration or student grading, advanced exception handling, and server-side logging to help administrators quickly diagnose and resolve issues. Additionally, graceful recovery strategies ensure minimal service disruptions, providing clear and helpful feedback to students and instructors whenever problems arise.

Maintainability

The platform emphasizes maintainability through comprehensive documentation, utilizing tools such as GitHub and clear internal comments to ensure transparency and facilitate future updates, with strict version control maintained throughout the development lifecycle. Code readability is prioritized by adhering to clean-code standards, including detailed inline comments on functions and classes, making the system easier to understand and maintain. Error monitoring is implemented through local server log tracking and standardized error codes, enabling timely issue resolution and maintaining system stability. Additionally, the platform is designed for scalability, allowing it to handle a growing number of students and academic departments without compromising performance.

3.1 Architectural Design

The ELMS system is structured as a mobile-based application following a three-tier architecture model, which enhances scalability, maintainability, and performance by organizing the system into three distinct layers:

1. Presentation Layer (Client-Side UI)

The Presentation Layer serves as the primary interface for the system, handling all direct student and instructor interactions through a cross-platform mobile application developed with the Flutter framework. This layer is responsible for rendering the user interface, managing local state, and ensuring a responsive experience across various devices. To maintain data flow and system functionality, it communicates with the application layer via secure API requests, ensuring that all user actions such as submitting an assignment or viewing a grade are transmitted safely and processed efficiently by the backend.

2. Application Layer (Business Logic & API Server)

The Application Layer serves as the central brain of the system, processing all incoming requests from the mobile client and executing the core academic business logic. This includes complex operations such as the algorithm for automated student grouping, grade calculations, and the management of secure user sessions. Functioning as a critical middleware between the presentation and data layers, it ensures that all interactions undergo strict data validation and enforces essential security policies, such as Role-Based Access Control (RBAC), to maintain the integrity and confidentiality of the university's data.

3. Data Layer (Database Server & Local Integrations)

The Data Layer serves as the foundational repository for the system, taking full responsibility for data management, secure storage, and efficient retrieval on the university's local infrastructure. It encompasses the Local SQL Database server, which stores structured information such as user credentials and course metadata, alongside integrated connections to local storage for hosting high-volume assets. This includes the physical storage of lecture materials, student assignment files, and comprehensive academic records, ensuring that all digital assets are organized and accessible to the upper layers of the system.

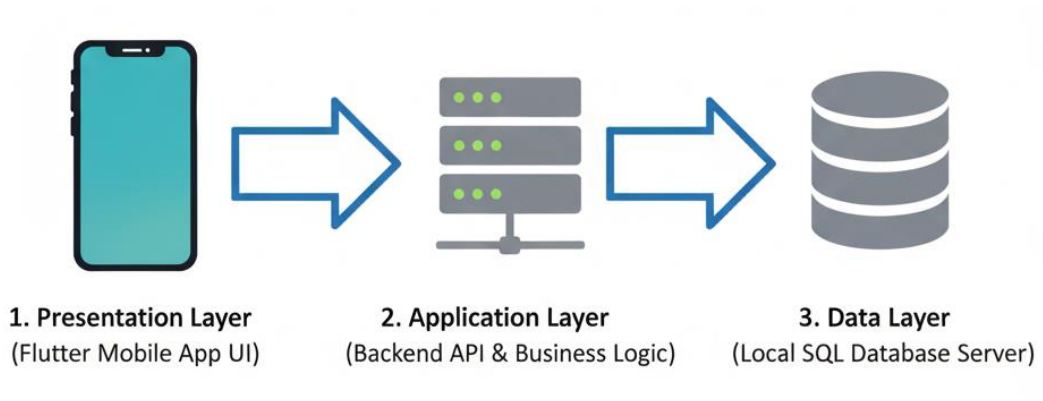

Figure 3.1: ELMS Three-Tier Architecture Diagram

Subsystem

System decomposition is a fundamental aspect of system design, enabling the breakdown of a complex system into smaller, more manageable subsystems. This structured approach enhances system comprehensibility, maintainability, and scalability by organizing related functionalities into distinct modules.

Each subsystem is a self-contained unit consisting of classes, associations, operations, events, and constraints, all working together to fulfill specific responsibilities. These subsystems are interconnected, ensuring smooth interactions while maintaining a cohesive system architecture.

The subsystems of ELMS are as follows:

- **User Interface Subsystem:** Manages the Flutter-based mobile screens and user interactions for students, instructors, and admins.
- **User Management Subsystem:** Handles registration, authentication (login), and role-based access control for the university community.
- **Course and Resource Subsystem:** Manages the uploading, storage, and retrieval of lecture materials, such as PDFs and video content.
- **Assessment and Grading Subsystem:** Handles the creation of quizzes, assignment submissions, and the processing of student grades.
- **Automated Grouping Subsystem:** Contains the logic for automatically organizing students into project or study groups based on specific criteria.
- **Data Management Subsystem:** Manages the local SQL database operations, including data persistence, backups, and retrieval of academic records.

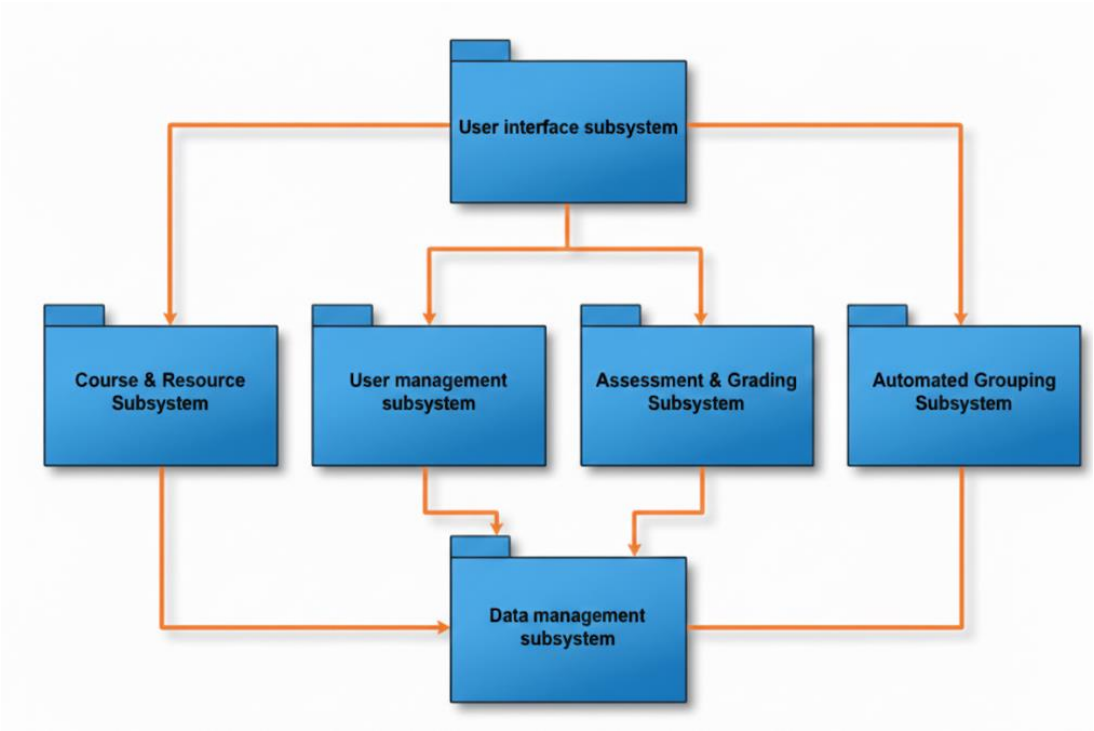

Figure 3.2: ELMS Subsystem Model – Modern Structure

3.1.1 Component Modeling

In a component-based architecture, the ELMS is divided into smaller, self-contained units known as components. Each component is designed to handle a specific set of academic tasks or services within the system, contributing to the overall functionality in a modular and maintainable way. These components are loosely coupled, meaning they are designed to function independently, allowing for flexibility, scalability, and easier maintenance within the university's local server environment.

However, while each component operates autonomously, it interacts with other components to perform tasks that require collaboration. These interactions occur through well-defined APIs or internal service calls, ensuring that data flows seamlessly across components. This design allows ELMS to manage a variety of tasks such as course material distribution, automated student grouping, user authentication, and grade management without any single component being overly dependent on others.

User Interface Component

The user interface subsystem allows interaction with the other subsystems. It facilitates the interaction of end users (students, instructors, and administrators) with the core functionalities of the system.

The interface subsystem includes the following interfaces:

- **User Management Subsystem Interface:** This interface enables users of the system to interact with the user management subsystem for registration, profile updates, and secure login.
- **Data Management Subsystem Interface:** This interface allows interfacing with the data management subsystem to view or update academic records stored in the local SQL database.
- **Assessment and Grading Interface:** This interface enables students to submit assignments and instructors to record, calculate, and publish student grades.
- **Course Content and Resource Interface:** This interface provides a secure portal for instructors to upload lecture materials (PDFs/Videos) and for students to view or download them.
- **Automated Grouping Interface:** This interface manages the configuration and viewing of student groups, allowing instructors to set grouping criteria and students to see their assigned project teams.
- **Notification and Announcement Interface:** This interface manages the delivery of campus-wide alerts, schedule changes, and academic deadlines to the users' mobile devices.

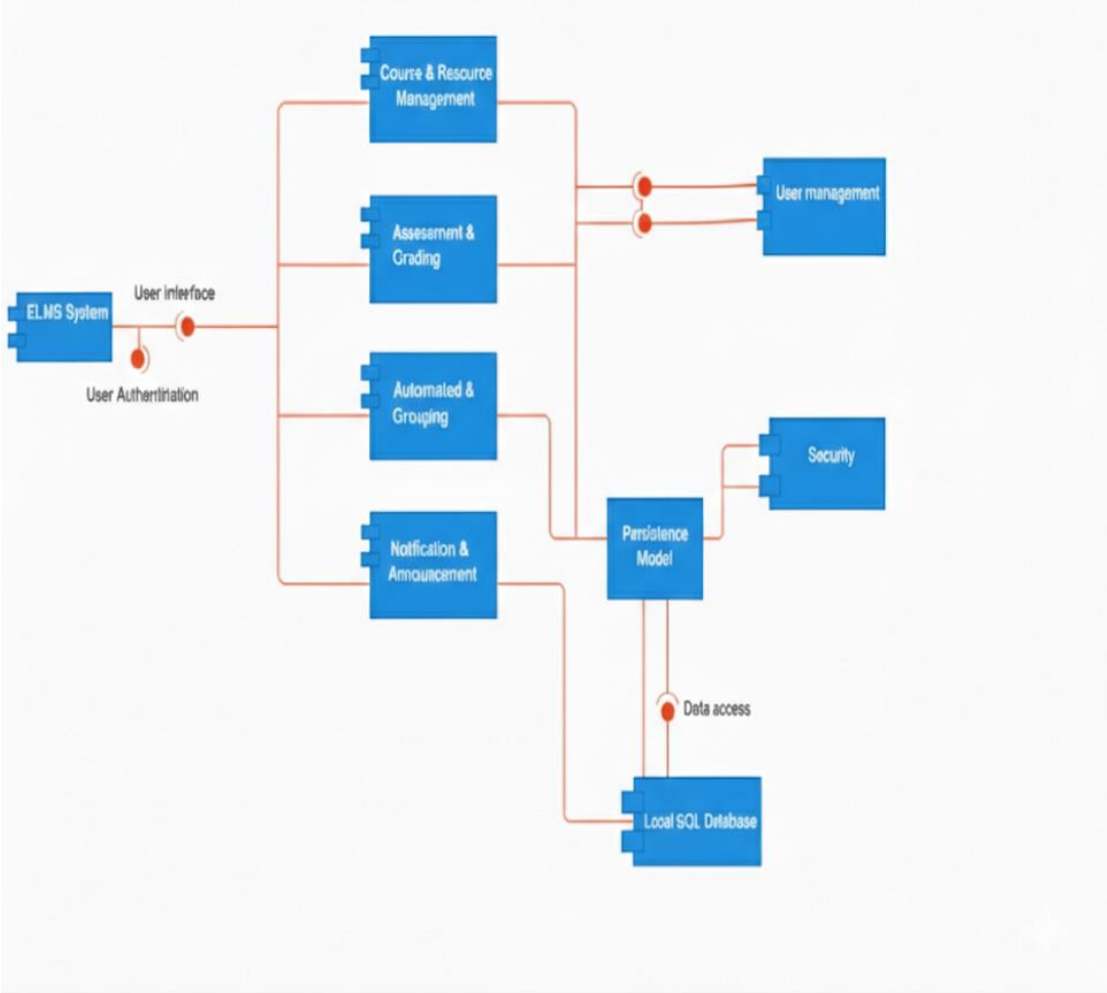

Figure 3.3: Component Diagram of ELMS System

3.1.2 Deployment Modeling

The ELMS system is deployed using a three-tier architecture model, distributing components across local university infrastructure and mobile devices to ensure scalability, maintainability, and performance. Each layer is deployed on dedicated hardware, ensuring optimized resource allocation and security within the campus network.

1. Client Layer (Presentation Layer Deployment)

Client Devices (Mobile):

The Client Layer represents the primary entry point for all users of the system, facilitating direct interaction through a dedicated ELMS Mobile Application designed for installation on both Android and iOS devices. This layer is responsible for the user interface and overall experience, ensuring that students and instructors can navigate their academic tasks efficiently. When a user performs an action such as logging in or accessing a course the application transmits secure

requests to the local Application Server using standardized API protocols. Developed with the Flutter framework and the Dart programming language, this layer provides a high-performance, cross-platform solution that maintains a consistent look and feel while leveraging local hardware for optimal responsiveness.

Environment:

The system is designed for a streamlined deployment environment where the mobile application is distributed directly to user devices. This is achieved through standard app packages, such as .apk for Android and .ipa for iOS, or via internal university distribution channels to guarantee widespread accessibility across the student body. By utilizing these direct distribution methods, the university ensures that all students and instructors can easily install and update the platform on their personal hardware, bypassing the need for public app store dependencies and maintaining a controlled, secure academic network.

2. Application Layer (Business Logic & API Server Deployment)

Application Server:

The Application Server serves as the central hub for the ELMS, hosting the backend API and the core business logic required for academic operations. It is responsible for processing user authentication, managing the complex algorithms for automated student grouping, and coordinating the distribution of digital learning resources to the mobile clients. Furthermore, the server manages all internal communication with the local data layer, ensuring that information flows securely and accurately between the database and the user interface. To protect the system, it incorporates a comprehensive suite of security features, including local firewall protection, rigorous session management, and strict API request validation to prevent unauthorized system access.

Hosting Environment: Deployed on a dedicated local server machine within the university's data center to ensure low latency and data sovereignty.

3. Data Layer (Database Server & Local Storage Deployment)

Database Server:

The Database Server serves as the permanent storage engine for the ELMS, running a Local SQL Server such as MySQL or PostgreSQL to manage the university's critical academic data. It is responsible for housing comprehensive student profiles, detailed academic records, and complex course enrollment data. By utilizing structured queries, the server ensures efficient data handling and maintains high-integrity storage for student grades and assessment results. This centralized repository is optimized for speed and reliability, providing a secure foundation that allows the application layer to retrieve and update academic information with precision and consistency.

Local Storage: Dedicated server storage for hosting large educational assets such as lecture videos and PDF resources, ensuring fast retrieval over the local area network (LAN).

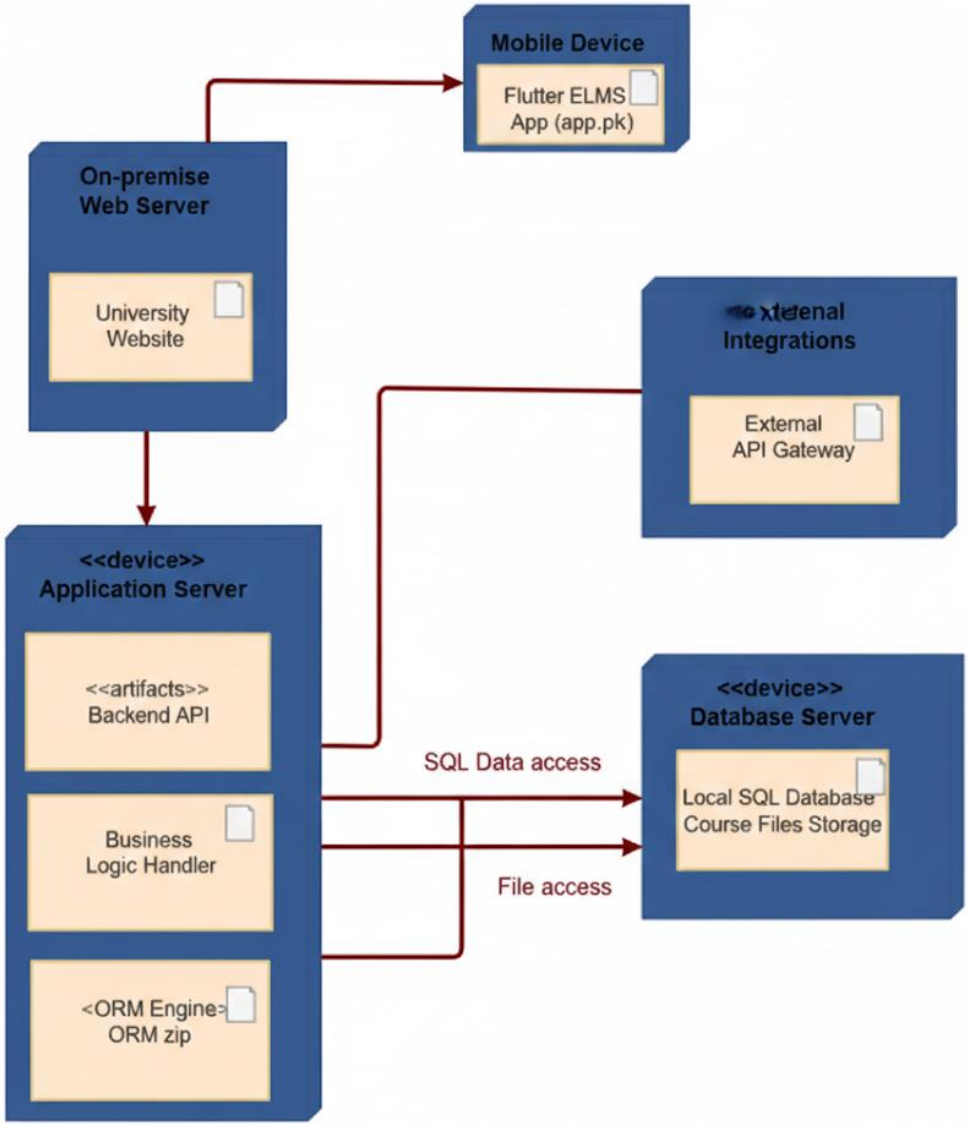

Figure 3.4: Deployment Diagram of ELMS System

Communication Flow

The communication flow begins with the mobile device, which sends user requests such as login attempts or assignment submissions via the Flutter application to the local server. These requests are first handled by the API Gateway, which forwards the business logic requirements to the Application Server. The Application Server performs the necessary computations, including automated student grouping, grade calculations, and user role verification. For file transfers, such as lecture videos or PDFs, the Application Server interacts with the local file storage service to manage uploads and downloads. When academic records, user credentials, or course data are needed, the Application Server communicates with the local SQL database server to retrieve or

store the relevant information. Once processing is complete, the response whether data or confirmation is sent back through the same path, ultimately reaching the Flutter UI on the mobile device.

Justification for the Architecture

The deployment design of the ELMS adopts a structured multi-tier architecture, offering several advantages tailored to the university environment. Its modular design allows the system to be divided into distinct layers, enabling modifications to the Flutter UI without impacting the underlying SQL database structure. The architecture also supports scalability, as the local server can be upgraded with additional RAM or storage to accommodate a growing student population, ensuring the system evolves alongside the university's needs. By utilizing a local SQL server, the university retains full control over sensitive student data, enhancing data sovereignty and security while minimizing the risk of breaches through restricted external access. Additionally, hosting the server and database on the campus network optimizes performance by reducing internet latency, allowing students to access and download large educational materials more quickly and efficiently.

3.2 Detail Design

3.2.1 Design Class Model

The Design Class Model represents the object-oriented structure of the ELMS, defining the classes, attributes, methods, relationships, access visibility, and multiplicity. This model serves as the blueprint for the implementation phase, ensuring a clear understanding of how academic entities such as students, courses, and grades interact within the local system.

Each class in the system is described with:

Attributes: Data members such as StudentID, CourseName, or GradeValue, including their data types and access modifiers (e.g., private for sensitive records, public for course descriptions).

Methods: Functions associated with the class, such as calculateGPA(), submitAssignment(), or generateGroups(), specifying return types and parameters.

Relationships: Structural connections between classes, including Inheritance (e.g., both Students and Instructors inheriting from a User base class) and Associations (e.g., a Student "enrolled in" a Course).

Multiplicity: Defines the numerical relationship between instances, such as one Instructor managing many (1..*) Courses, or one Student belonging to exactly one (1) Group for a specific project.

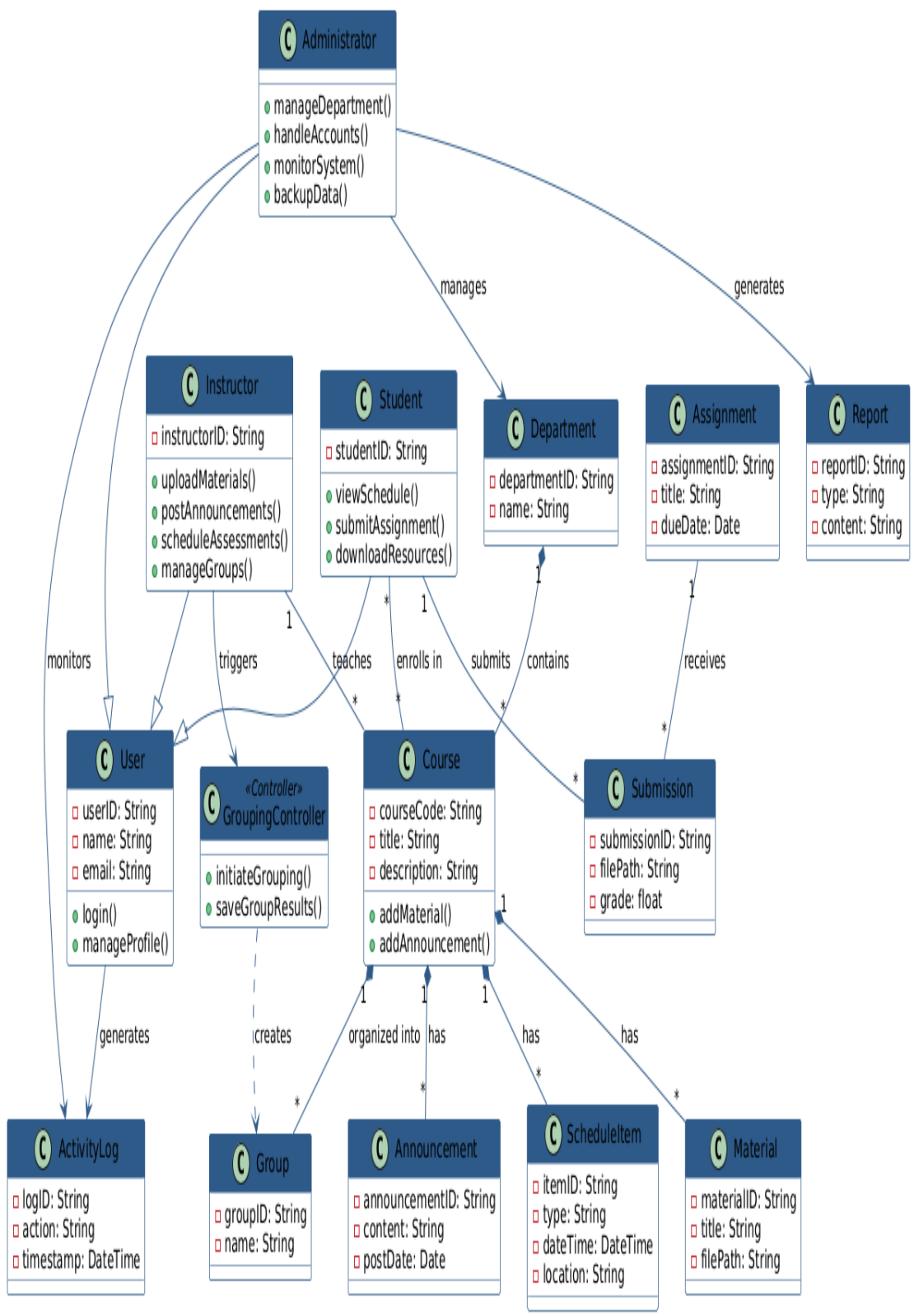

Figure 3.5: Class Diagram of ELMS System

Table 3.1: Administrator Entity Description

Attribute Name	**Data Type**	**Length**	**Constraint**
AdminID	String	30	PRIMARY KEY
FullName	String	50	NOT NULL
Email	String	40	NOT NULL
Password	String	20	NOT NULL

Table 3.2: Instructor Entity Description

Attribute Name	**Data Type**	**Length**	**Constraint**
InstID	String	30	PRIMARY KEY
FName	String	40	NOT NULL
LName	String	40	NOT NULL
DeptID	String	30	FOREIGN KEY
Email	String	40	NOT NULL
Password	String	20	NOT NULL

Table 3.3: Student Entity Description

Attribute Name	**Data Type**	**Length**	**Constraint**
StudentID	String	30	PRIMARY KEY
FName	String	40	NOT NULL
LName	String	40	NOT NULL
DeptID	String	30	FOREIGN KEY
GroupID	String	30	FOREIGN KEY (Optional)

Table 3.4: Department Entity Description

Attribute Name	Data Type	Length	Constraint
DeptID	String	30	PRIMARY KEY
DeptName	String	50	NOT NULL

Table 3.5: Course Entity Description

Attribute Name	Data Type	Length	Constraint
CourseID	String	30	PRIMARY KEY
Title	String	100	NOT NULL
Description	Text	-	NOT NULL
DeptID	String	30	FOREIGN KEY
InstID	String	30	FOREIGN KEY

Table 3.6: Material Entity Description

Attribute Name	Data Type	Length	Constraint
MaterialID	String	30	PRIMARY KEY
Title	String	50	NOT NULL
FilePath	String	255	NOT NULL
CourseID	String	30	FOREIGN KEY

Table 3.7: Assignment Entity Description

Attribute Name	Data Type	Length	Constraint
AssignID	String	30	PRIMARY KEY
Title	String	50	NOT NULL
DueDate	DateTime	-	NOT NULL
CourseID	String	30	FOREIGN KEY

Table 3.8: Group Entity Description

Attribute Name	Data Type	Length	Constraint
GroupID	String	30	PRIMARY KEY
GroupName	String	50	NOT NULL
CourseID	String	30	FOREIGN KEY

Table 3.9: Activity Log Entity Description

Attribute Name	Data Type	Length	Constraint
LogID	String	30	PRIMARY KEY
UserID	String	30	FOREIGN KEY
Action	String	100	NOT NULL
Timestamp	DateTime	-	NOT NULL

3.2.2 Persistent Model

The persistent model represents how the system's data will be stored in an object-oriented database. Each entity in the ELMS is modeled as a class, with its attributes, data types, length constraints, and relationships to other classes clearly defined. The relationships include associations, multiplicities, and roles, ensuring that object references can model real-world academic structures like courses, students, and instructors.

Class Definitions

1. **Administrator Class**

Represents the system administrators responsible for managing users and system settings.

Relationships:

- No direct associations; administrators manage system operations.

Multiplicity:

- 1 system has 1..* administrators.

Table 3.10: Administrator Class

Attribute Name	Data Type	Length	Constraint
adminID	String	30	Primary Key
fullName	String	50	Not Null
email	String	40	Not Null
password	String	20	Not Null

2. **Instructor Class**

Represents instructors who manage courses and upload materials.

Relationships:

- **Department**: Each instructor belongs to one department (1..1).
- **Course**: An instructor can manage multiple courses (1..*).

Roles:

- instructor role in Course association.

Table 3.11: Instructor Class

Attribute Name	Data Type	Length	Constraint
instID	String	30	Primary Key
fName	String	40	Not Null
lName	String	40	Not Null
deptID	String	30	Foreign Key → Department
email	String	40	Not Null
password	String	20	Not Null

3. **Student Class**

Represents students enrolled in courses.

Relationships:

- **Department**: Each student belongs to one department (1..1).
- **Group**: Each student can optionally belong to one group (0..1).

- **Course**: Students enroll in multiple courses (0..*).

Roles:

- member in Group association.

Table 3.12: Student Class

Attribute Name	**Data Type**	**Length**	**Constraint**
studentID	String	30	Primary Key
fName	String	40	Not Null
lName	String	40	Not Null
deptID	String	30	Foreign Key → Department
groupID	String	30	Foreign Key → Group (Optional)

4. **Department Class**

Represents university departments managing courses and instructors.

Relationships:

- **Instructor**: One department has 0..* instructors.
- **Student**: One department has 0..* students.
- **Course**: One department manages 0..* courses.

Table 3.13: Department Class

Attribute Name	**Data Type**	**Length**	**Constraint**
deptID	String	30	Primary Key
deptName	String	50	Not Null

5. **Course Class**

Represents courses offered by the university.

Relationships:

- **Department**: Each course belongs to one department (1..1).

- **Instructor**: Each course is managed by one instructor (1..1).
- **Material**: A course has 0..* materials.
- **Assignment**: A course has 0..* assignments.
- **Group**: A course can have 0..* student groups.

Table 3.14: Course Class

Attribute Name	Data Type	Length	Constraint
courseID	String	30	Primary Key
title	String	100	Not Null
description	Text	-	Not Null
deptID	String	30	Foreign Key → Department
instID	String	30	Foreign Key → Instructor

6. **Material Class**

Represents lecture materials (PDFs, videos) for courses.

Relationships:

- **Course**: Material belongs to one course (1..1).

Table 3.15: Material Class

Attribute Name	Data Type	Length	Constraint
materialID	String	30	Primary Key
title	String	50	Not Null
filePath	String	255	Not Null
courseID	String	30	Foreign Key → Course

7. **Assignment Class**

Represents assignments for courses.

Relationships:

Course: Assignment belongs to one course (1..1).

Table 3.16: Assignment Class

Attribute Name	Data Type	Length	Constraint
assignID	String	30	Primary Key
title	String	50	Not Null
dueDate	DateTime	-	Not Null
courseID	String	30	Foreign Key → Course

8. **Group Class**

Represents student groups for projects or assignments.

Relationships:

- **Course**: Group belongs to one course (1..1).
- **Student**: A group can have 1..* students.

Roles: member role in Student association.

Table 3.17: Group Class

Attribute Name	Data Type	Length	Constraint
groupID	String	30	Primary Key
groupName	String	50	Not Null
courseID	String	30	Foreign Key → Course

9. **ActivityLog Class**

Represents actions performed by users (students, instructors, admins).

Relationships:

- **User**: Each activity log belongs to one user (1..1).

Table 3.18: ActivityLog Class

Attribute Name	**Data Type**	**Length**	**Constraint**
logID	String	30	Primary Key
userID	String	30	Foreign Key → User (Admin/Instructor/Student)
action	String	100	Not Null
timestamp	DateTime	-	Not Null

Relationships Overview

Table 3.19: Class Relationships

Class A	**Class B**	**Relationship Type**	**Multiplicity**	**Role**
Instructor	Department	Association	1..* ↔ 1	instructor
Student	Department	Association	0..* ↔ 1	student
Course	Instructor	Association	1..* ↔ 1	instructor
Course	Department	Association	0..* ↔ 1	department
Material	Course	Association	0..* ↔ 1	material
Assignment	Course	Association	0..* ↔ 1	assignment
Group	Course	Association	0..* ↔ 1	group
Student	Group	Association	0..* ↔ 1	member
ActivityLog	User	Association	0..* ↔ 1	actor

E-R Diagram

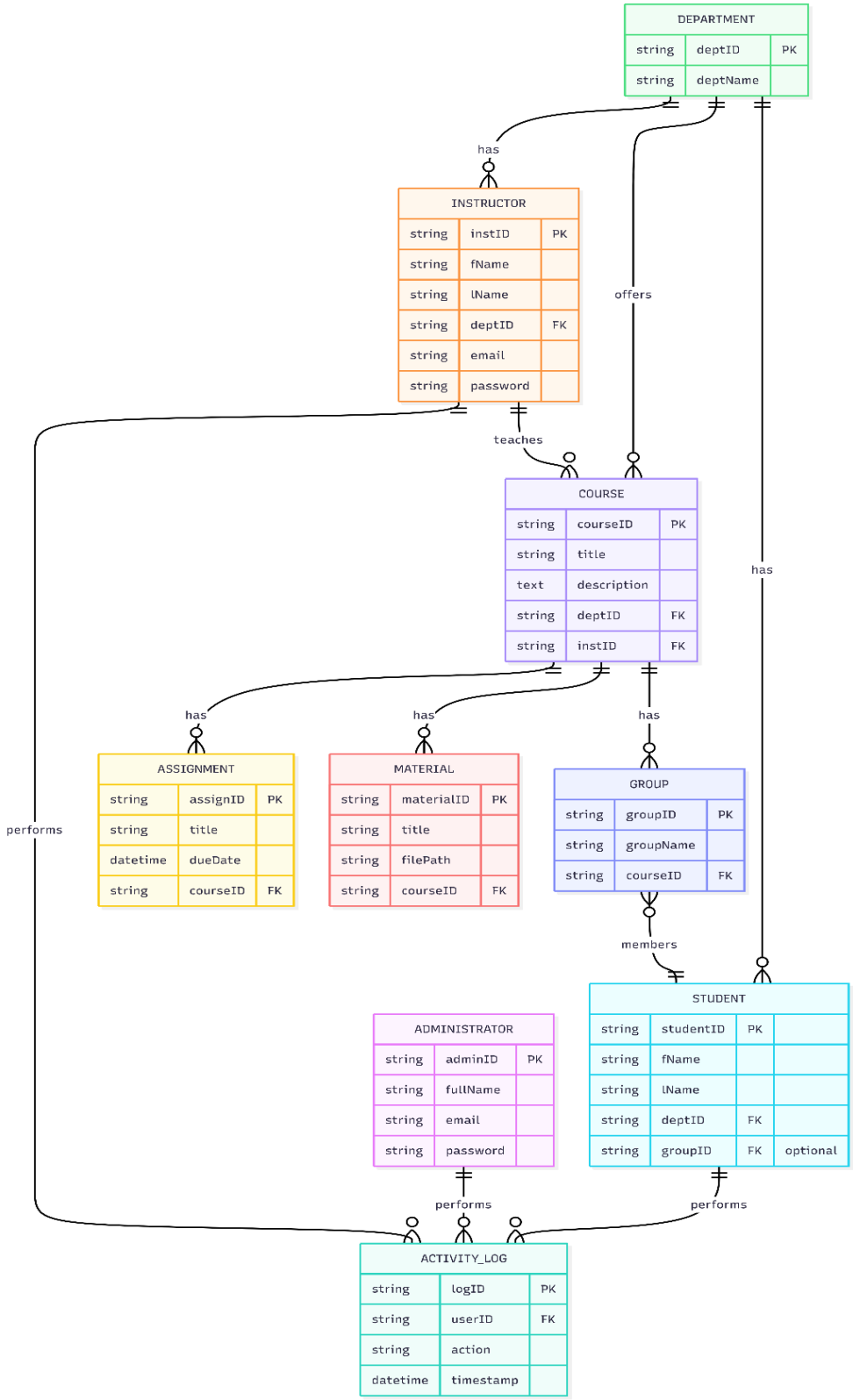

Figure 3.6: E-R Diagram of ELMS System

3.4 Access Control and Security

The E-Learning Management System implements robust access control and security mechanisms to ensure that different actors (users) have appropriate permissions based on their roles. Security measures are enforced to protect sensitive academic data, prevent unauthorized grade changes, and maintain the integrity of student records.

Table 3.20: Access Control across Users

Tasks	**Admin**	**Instructor**	**Student**
Manage User Accounts (Create/Update/Delete)	✓		
Course Creation & Resource Upload		✓	
Course Enrollment			✓
Access Learning Materials (PDF/Video)		✓	✓
Submit Assignments			✓
Grade Assignments & Provide Feedback		✓	
Manage Automatic Student Grouping		✓	
Post System-Wide Announcements	✓		
Post Course-Specific Announcements		✓	
View Own Grade Reports			✓
System Maintenance & Data Backups	✓		
Update Personal Profile	✓	✓	✓
Chat & Discussion Forum Interaction	✓	✓	✓